
Mechanistic Interpretability with Known Ground Truth: Benchmarking Sparse Autoencoders on the Random Hierarchy Model

Tigrane Ponsin

Department of Physics

École Normale Supérieure PSL

Paris, France

tigrane.ponsin@ens.psl.eu

July 2026



Département
de Physique

École normale
supérieure

EPFL

Internship report. Supervised by Prof. Matthieu Wyart, PCSL Laboratory, EPFL

Abstract

Sparse autoencoders (SAEs) and Circuit Tracing have emerged as promising tools for uncovering semantic concepts and the mechanistic behavior of language models. However, these interpretability methods rely on a human or a language model to label nodes and group them into understandable circuits, which raises skepticism on the robustness of the findings. In the case of language, while it is already hard to assess the faithfulness of the recovered circuits to the network’s computation, there is also no reference to what the recovered features and circuits should logically be since there is no unique ground truth structure of language. This motivates benchmarking these methods by studying them on synthetic data with established latent structure. We thus chose to analyze transformers trained on the Random Hierarchy Model (RHM), a probabilistic context-free grammar whose latent variables and generative tree are known, which lets us label every SAE feature automatically via the conditional entropy of the generative latents given feature activation. Sweeping the coefficient controlling the sparsity of SAE features reveals an onset of error beyond which inserting the SAE into the model degrades its generalization performance. Just before it, features become highly selective for the latents. Combining feature analysis, ablation experiments, and attribution-graph circuit tracing, we examine both the behavior of SAEs and how it relates to the way the model dispatches information across positions and depth. We propose an automatic, rather than hand-picked, grouping of circuit features, and we show examples in which the recovered circuit matches the generative tree, though many circuits remain more complex.

1 Introduction

Large language models now handle grammar, reasoning, and broad world knowledge with a generality that was hard to anticipate [1, 2]. Yet we know how to build these models far better than we know what goes on inside them. They are complex non-linear systems composed of an enormous number of interacting elements. Mechanistic interpretability aims to uncover what models do through understandable mechanisms [3]. This serves many goals, including deepening our scientific understanding of how these machines achieve such performance, supporting AI safety by enabling direct audits and predictions of model behavior, or improving trust in model decisions in medical domains, to name a few.

One response is to reverse-engineer a trained model component by component, identifying groups of them corresponding to specific concepts or behaviors and building connections between them by evaluating their interactions [4]. While it has recovered genuine mechanistic explanations, such as the induction heads that drive in-context learning [5], network components often make poor units of meaning [6].

Sparse Autoencoders and circuit tracing are now primary mechanistic interpretability tools for understanding neural activations and turning them into human-readable outputs. SAEs decompose neural activation into directions and use a few of them at a time to reconstruct their input, hence the name "sparse" [7, 8]. These directions, SAE features, often track single, human-interpretable concepts, pulling them out of superposition in activation space [9, 10]. Circuit tracing builds causal graphs using features as nodes [11]. Anthropic’s interpretability team has shown impressive semantically meaningful graphs using sparse transcoders (conceptually similar to SAEs) [12, 13]. However, there is a basic difficulty in evaluating their methods, since in the language setting there is no ground truth for what the recovered features and circuits should be. Features are labeled by humans, and circuits are simplified by hand-selecting and grouping features. This makes it hard to tell how robust and faithful to the model’s internal computations these methods are and hard to set the hyperparameters in a principled way. To progress in that direction, we propose working in a setting where the ground-truth generative process is known in advance, so that features and circuits can be checked against it.

We turn to the Random Hierarchy Model (RHM), a synthetic generative model of data whose latent tree is fully known. It captures the compositional structure attributed to language, images, and other natural data, while remaining fully tractable [14–16]. Because the tree and its latent variables are known, features can be labeled by quantitatively measuring how much they tell us about each latent variable, with no human judgment involved, and the recovered circuit can be compared with the generative circuit that actually produced the data. In this report, we evaluate SAE features and the resulting circuit tracing for transformers trained to classify the root. We have three main findings:

- **Setting the sparsity coefficient:** by sweeping over the sparsity coefficient λ , we find a threshold beyond which SAE reconstruction degrades substantially, which we denote as the onset of error. Slightly prior to that threshold, we find highly interpretable features.
- **Systematic labeling of features:** we introduce an entropy-based metric that enables automatic labeling. It allows us to analyze how information is displaced across the network and to identify evidence of overspecific features, a phenomenon called absorption. This method highlights the difference between transformers with and without residual skip connections. Additionally, it allows us to identify a dead-token phenomenon between the first two blocks in the network.
- **Circuit recovery:** by adapting Anthropic’s local replacement model to our SAEs, we construct circuits and introduce an automatic grouping procedure that replaces hand selection. We discuss its performance across multiple RHM instances and showcase perfectly recovered ground-truth trees in certain settings.

The code for this project is available at <https://github.com/tigranepson/SAE-on-RHM>.

2 Background

2.1 The Random Hierarchy Model

The Random Hierarchy Model (RHM), introduced by [14], is a synthetic generative model of hierarchically structured data. Formally, it is a probabilistic context-free grammar [17] with randomly drawn production rules. Though variants exist [18], we restrict ourselves to a root classification task on the original fixed-tree formulation. Fig. 1 provides a visual representation of what is described below.

Tree structure and notation. A datum of the RHM is a labeling of a fixed regular tree of depth L and branching factor s . The tree has s^L leaves and $L + 1$ levels: level 0 at the leaves, level L at the root. To each level $\ell \in \{0, \dots, L\}$ we associate a vocabulary $\mathcal{V}_\ell$ of size v , and we denote the node at position u of level ℓ by $h_u^{(\ell)} \in \mathcal{V}_\ell$. The root $h_0^{(L)}$ plays the role of the class label, the leaves $h_0^{(0)}, \dots, h_{s^L-1}^{(0)}$ form the visible sequence, and the intermediate nodes are latent variables.

Production rules and synonyms. The grammar consists of one production rule per level. At each level $\ell \in \{1, \dots, L\}$, every parent symbol $a \in \mathcal{V}_\ell$ is associated with exactly m distinct s -tuples in $\mathcal{V}_{\ell-1}^s$ that correspond to *synonyms*. The vm grammatical tuples at level $\ell - 1$ are drawn uniformly without replacement from the v^s possible s -tuples, then partitioned uniformly into v groups of size m , one per parent. We require $vm \leq v^s$ at every level, with $m = 1$ corresponding to a trivial deterministic grammar and $m = v^{s-1}$ saturating the bound (every s -tuple becomes grammatical).

A central structural assumption is *non-ambiguity*: the synonymic groups of different parents are disjoint, so every grammatical tuple has a unique parent. Each level- ℓ rule can therefore be summarized by an injective inverse map $g_\ell : \mathcal{S}_{\ell-1} \rightarrow \mathcal{V}_\ell$, where $\mathcal{S}_{\ell-1} \subset \mathcal{V}_{\ell-1}^s$ is the set of vm grammatical tuples. Two tuples $\nu, \nu' \in \mathcal{S}_{\ell-1}$ are synonyms iff $g_\ell(\nu) = g_\ell(\nu')$.

Generative process. A datum is generated top-down. The root $h_0^{(L)}$ is sampled uniformly from $\mathcal{V}_L$. Then, recursively from $\ell = L$ down to $\ell = 1$, every node $h_u^{(\ell)} = a$ is expanded into one of the m synonymic s -tuples of a , chosen uniformly at random; the resulting tuple is written into the s children $h_{us}^{(\ell-1)}, \dots, h_{(u+1)s-1}^{(\ell-1)}$. After L steps one obtains a fully labeled tree, of which only the leaf sequence $h_{0:s^L-1}^{(0)} \in \mathcal{V}_0^{s^L}$ is observed, and serves as input for our transformers.

2.2 Sparse Autoencoders

Features in superposition. A central hypothesis in mechanistic interpretability is that neural networks represent more features than they have neurons by storing them in *superposition*: features correspond to directions in activation space rather than to individual neurons [6]. A feature, therefore, cannot, in general, be read off a single neuron or coordinate of the hidden representation. Sparse autoencoders (SAEs) address this by learning an overcomplete dictionary of feature directions directly from the activations of a *frozen* model. The approach was introduced for language models by [7], [8] and [9].

Architecture. Let $x \in \mathbb{R}^{d_{\text{model}}}$ be an activation of the frozen model at some intermediate layer that we wish to decompose (an MLP activation in [9]; a residual-stream activation in [10]). An SAE is a single-hidden-layer autoencoder with an overcomplete hidden width $d_{\text{SAE}} > d_{\text{model}}$. The encoder maps x to non-negative feature activations $z \in \mathbb{R}^{d_{\text{SAE}}}$, and the decoder reconstructs x as a sparse combination of feature directions:

$$z = \text{ReLU}(W_{\text{enc}} x + b_{\text{enc}}), \quad (1)$$

$$\hat{x} = W_{\text{dec}} z + b_{\text{dec}}, \quad (2)$$

with $W_{\text{enc}} \in \mathbb{R}^{d_{\text{SAE}} \times d_{\text{model}}}$, $W_{\text{dec}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{SAE}}}$, $b_{\text{enc}} \in \mathbb{R}^{d_{\text{SAE}}}$, $b_{\text{dec}} \in \mathbb{R}^{d_{\text{model}}}$. The column $W_{\text{dec}}[:, i]$ is the direction feature i writes into activation space, and $z_i \geq 0$ mediates the norm of that vector.

Training objective. The SAE is trained post hoc on activations of the frozen model; the base model’s weights are never updated. It is trained to perform the identity function on its input while activating the fewest features in the hidden layer. This is enforced by a loss combining reconstruction with a sparsity penalty:

$$\mathcal{L} = \underbrace{\|x - \hat{x}\|_2^2}_{\text{reconstruction}} + \lambda \underbrace{\sum_{i=1}^{d_{\text{SAE}}} z_i \|W_{\text{dec}}[:, i]\|_2}_{\text{sparsity}}. \quad (3)$$

The sparsity term is an ℓ_1 penalty on the feature activations [19], which drives most features to be inactive on any given input so that x is explained by a small subset of (ideally monosemantic) directions. Weighting each feature’s penalty by its decoder-column norm $\|W_{\text{dec}}[:, i]\|_2$ removes the degenerate solution of shrinking z while scaling up W_{dec} . The coefficient λ sets the reconstruction–sparsity trade-off.

3 Setup

Data. All experiments use binary trees ($s = 2$). Transformers are trained for root classification on P samples chosen so that $P^* \ll P \ll P_{\text{max}}$, where $P^* \sim v m^L$ is the sample complexity of [14] and $P_{\text{max}} = v m^{(s^L-1)/(s-1)}$ is the total number of data the grammar generates: the models generalize, but we can look at the test loss or accuracy to witness they don’t memorize. The configurations studied are summarized in Tab.1, App.A.

Architecture. The transformer has L pre-LayerNorm blocks, one per RHM level, with bidirectional 8-head attention and ReLU MLPs, at $d_{\text{model}} = 512$. We use learned token and position embeddings, so position u carries leaf u throughout. A final LayerNorm, mean pooling collapsing the s^L positions into one vector, and a linear classifier produce the v logits. We train transformers both with and without residual connections. In the absence of an identity path, every block in the non-residual variant rewrites the representation it receives, allowing the architecture to better match the coarse-graining topology of the RHM. Models are trained with AdamW using cross-entropy until the training loss reaches 10^{-3} ; the lowest test-loss checkpoint is used for all downstream work. Parametrization and hyperparameters are given in App.A.

Sparse autoencoders. We use one SAE per transformer block trained independently on the post-block stream with every position contributing (s^L vectors per sample), at width $d_{\text{SAE}} = 4 d_{\text{model}}$. We also occasionally trained *pooled* SAEs on the post-LayerNorm pooled vector, the exact input of the linear classifier. SAEs are trained, and all evaluations run, on samples whose seeds are disjoint from the transformer’s training data. Optimization details are in App.A.

Notations. Fig. 1 summarizes the setup and notations explained below. Consider a datum generated by the RHM, whose leaves $h_0^{(0)}, \dots, h_{s^L-1}^{(0)}$ are fed into the model. For each position $u \in \{0, \dots, s^L - 1\}$, let $x_u^{(0)}$ denote the embedding of leaf $h_u^{(0)}$; these are the vectors passed into the first transformer block. For $k \geq 1$, the blocks act as

$$(x_0^{(k)}, \dots, x_{s^L-1}^{(k)}) = \text{Block}_k(x_0^{(k-1)}, \dots, x_{s^L-1}^{(k-1)}),$$

so that $x_u^{(k)}$ is the output of Block_k at position u , and the network applies $\text{Block}_1, \dots, \text{Block}_L$ in sequence. The final representations are mean-pooled into

$$\tilde{x}^{(L)} = \frac{1}{s^L} \sum_{u=0}^{s^L-1} \text{LN}(x_u^{(L)}),$$

which is the vector fed to the linear classifier. Sparse autoencoders in between each transformer block reconstruct the activation vectors:

$$\hat{x}_u^{(k)} = \text{SAE}_k(x_u^{(k)}).$$

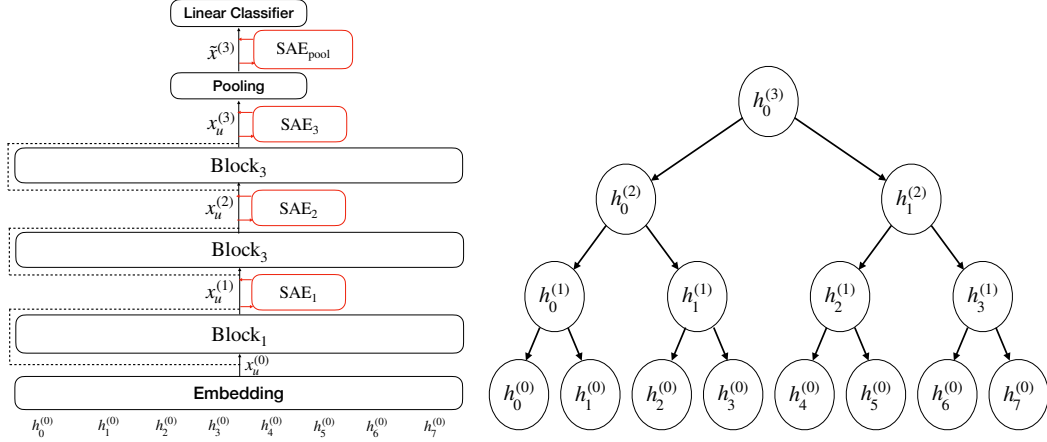


Figure 1: **Model architecture and RHM topology.** Example in the case of RHM parameters $L = 3$, $s = 2$. Left: Transformer with SAE insertions. Dashed lines represent optional residual connections. Right: RHM with notations.

4 SAE feature analysis

4.1 Simple sweep metrics

The SAE carries a free knob, the sparsity coefficient λ . To select it, we sweep over it for every layer and variant. While sweeps are routine for SAEs on language models [20], they are cheap and “sharp” in our case. Since the downstream task is root classification with a known random baseline, we can clearly identify whether inserting the SAE in the model breaks the network’s computation. The SAE fails if its reconstruction error is too large, causing the classification test error to spike. In the case of classification, it is a precise event rather than a diffuse rise in cross-entropy for language models.

For each λ we evaluate the trained SAE on a held-out analysis set and record:

- *Ever-active features*: the number of features that activate on at least one (token position, datum) of the analysis set.
- *Mean L_0* : the average number of features active on a single token position and datum.
- *Normalized classification error*: we insert the SAE in the network between its two blocks, classify the held-out set through the resulting model, and normalize by the random baseline, $\text{error}_{\text{SAE}} / (1 - \frac{1}{v})$.

The SAEs of interest sit slightly before the *onset of error*, which we define as the λ at which the normalized error first crosses 1% and lifts off its floor: the sparsest dictionaries that preserve the model’s predictions.

An example of these sweep metrics is shown in Fig. 2.

4.2 Feature–latent association

Entropy metrics. With the latents known, we score a feature by how much its firing tells us about each one. Treating feature i as the binary event $z_i > 0$, we estimate on a held-out analysis set, with the same number of samples as the transformer and SAE training sets, the conditional entropy of a latent $h_u^{(\ell)}$ given the feature is active,

$$H(h_u^{(\ell)} | z_i > 0) = - \sum_{a \in \mathcal{V}_\ell} P(h_u^{(\ell)} = a | z_i > 0) \log P(h_u^{(\ell)} = a | z_i > 0), \quad (4)$$

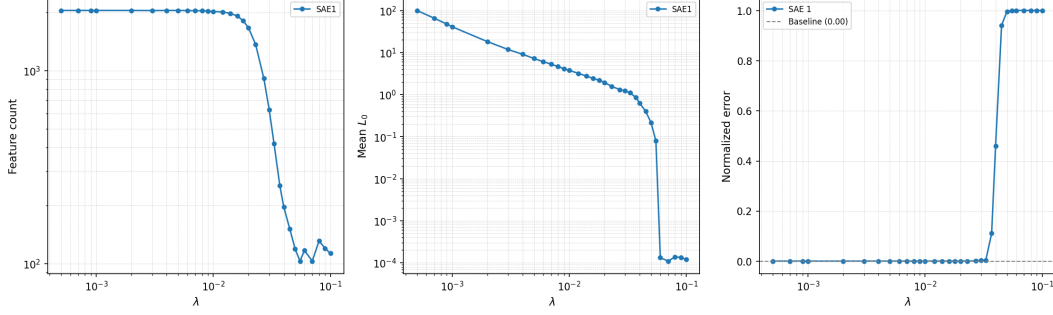


Figure 2: **Sweep metrics example.** The three metrics defined above (ever-active features, mean L_0 , normalized classification error) versus λ , for SAE_1 of a non-residual stream transformer trained on classification for RHM architecture $v = 16$, $L = 3$, $m = 16$. Close to the onset of error the L_0 is of order 1: activations are reconstructed from very few active features.

and normalize by the latent’s own entropy to obtain a normalized entropy in $[0, 1]$,

$$\eta_i(\ell, u) = \frac{H(h_u^{(\ell)} \mid z_i > 0)}{H(h_u^{(\ell)})} \quad (5)$$

$\eta_i = 0$ means the feature pins that latent’s value exactly, $\eta_i = 1$ that it says nothing. The denominators $H(h_u^{(\ell)})$ are obtained from the production rules of the RHM, so they are exact and not estimated. Latent marginals are close to uniform, so the denominators sit close to $\log v$, but the exact value depends on the sampled rule realization, which is why we normalize by it rather than by $\log v$.

Where the label may live. The transformer is as deep as the grammar: L blocks for the L levels of composition. This invites a step-for-step reading in which the two are aligned, block k composing its inputs by one level so that the stream after block k resolves the level- k latents, the first block recovering the leaves’ parents and the last recovering the root [14, 18]. We started our analysis with a locality intuition: $x_u^{(k)}$ would resolve its corresponding parent latent $h_{\lfloor u/s^k \rfloor}^{(k)}$. This corresponds to what we thought was the most “efficient” way to treat the classification task on the RHM with positional embeddings. We make a heuristic argument for it based on attention look-ups in App.B. However, based on observations from both feature analysis and early circuit tracing, we extended our analysis from simple locality. We use three different constraints for feature-latent association of SAE_k (Fig. 3):

- *Parent-constrained*: restricting the latent association to the parent latent $h_{\lfloor u/s^k \rfloor}^{(k)}$.
- *Level-constrained*: restricting the latent association to the corresponding level $h_j^{(k)}$, $j \in [0, s^{L-k} - 1]$.
- *Unconstrained*: no restriction on the latent level and position

Aggregate selectivity. To summarize a layer at a given sparsity for a given position, we average the labeled-latent score across active features on the analysis set. We first identify the label for each feature, restricting the $\arg \min$ search space based on the chosen constraint introduced previously:

$$(\ell_i, u_i) = \arg \min_{\ell, u} \eta_i(\ell, u) \quad (6)$$

In the case of parent-constrained, the $\arg \min$ is an artifact, we just pick the parent latent. Features contribute to the SAE’s reconstruction very unequally, so the average is weighted by their mean activation $\mathbb{E}[z_i]$; this also down-weights rarely-active features, whose conditional-entropy estimates are the noisiest. The averaged entropy then reads:

$$\bar{\eta} = \frac{\sum_i \mathbb{E}[z_i] \eta_i(\ell_i, u_i)}{\sum_i \mathbb{E}[z_i]} \quad (7)$$

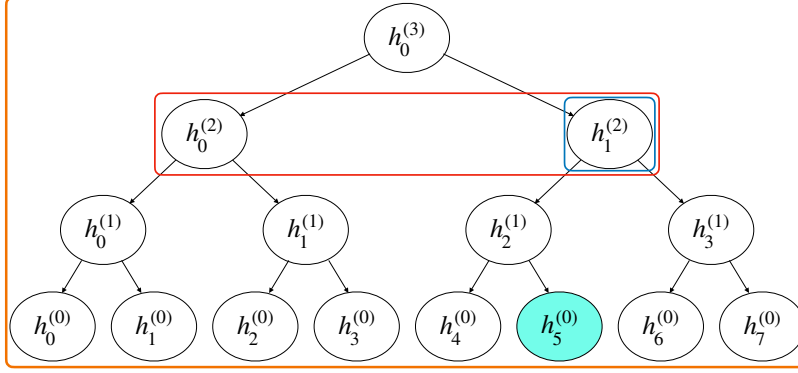


Figure 3: **RHM topology and the three search regions for feature-latent association.** Example for RHM parameters $L = 3$, $s = 2$, considering SAE_2 at token position 5. Parent-constrained corresponds to the **blue** box (the single parent latent), level-constrained to the **red** box (all level-2 latents), and unconstrained to the **orange** box (all latents and leaves).

Feature absorption and reassignment. An observation flagged a problem with unconstrained labeling. In some cases the weighted entropy $\bar{\eta}$ was already near 0 under level-constrained labeling, yet once the search was unconstrained a large fraction of features were relabeled *downward*, scoring even lower on a latent one level below (See Figs. 6 9, and 10). We read this as feature absorption, a behavior expected with hierarchical features [21].

Here *parent* and *child* are local and grammatical, distinct from the block-aligned parent latent of parent-constrained labeling: two latents at sequential levels tied by a single rule, a child $h_j^{(\ell)}$ and the latent $h_{\lfloor j/s \rfloor}^{(\ell+1)}$ that produces it. A dictionary could carry one feature for a and separate features for the child values. Absorption collapses this: the SAE learns one sharper feature for a child value c bundled with the parent a that produced it. It reads as a confident child detector while firing only inside a , so the unconstrained $\arg \min$ labels it a child feature though it pins the parent too tightly. Not every downward relabeling is pathological: a feature may *genuinely* track a child latent. The threshold α below is what separates these genuine child features from *absorbed* ones.

To flag this, we compare what the feature reveals about the parent against what its child value grammatically reveals. We condition on the *specific* value c_i the feature specialized to. Reading $H(h_{\lfloor j/s \rfloor}^{(\ell+1)} \mid h_j^{(\ell)} = c_i)$ from the production rules, we reassign a child-labeled feature to its parent when

$$\frac{H(h_{\lfloor j/s \rfloor}^{(\ell+1)} \mid z_i > 0)}{H(h_{\lfloor j/s \rfloor}^{(\ell+1)} \mid h_j^{(\ell)} = c_i)} < \alpha, \quad (8)$$

keeping the child as a tag, with α a chosen threshold value. This ratio is only defined when the latent assigned from the unconstrained labeling scheme has a parent, so it won't ever be computed and used to reassign a feature already labeled to a root value.

4.3 Experimental results on transformers without residual stream

In this section we analyze a transformer without residual stream trained for the RHM configuration $v = 16$, $L = 3$, $m = 16$. All metrics are computed on a held-out analysis set of the same size as the transformer and SAE training sets, keeping the full RHM realization of each sample so that latent values can be conditioned on. We run the held-out set through the model and the SAEs trained at each λ , record feature activations, and form the empirical distributions of latents conditioned on each feature firing that the analysis below uses.

For these experiments, we use $\alpha = 0.5$, chosen by sweeping over α for a λ value of interest (close to the onset of error), which revealed a plateau region in the proportion of reassigned features. We show

an example of this α sweep in Fig. 8a for SAE₂, which shows a clear distinction between genuine and absorbed features.

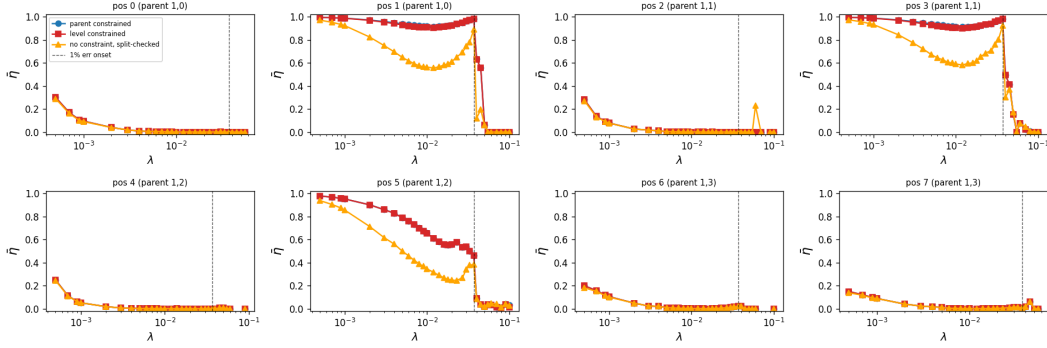


Figure 4: **Entropy curves for SAE₁, all token positions.** RHM parameters $v = 16, L = 3, m = 16$. Blue = parent-constrained (overlapping the red curve, locality assumption seems to hold), red = level-constrained, orange = unconstrained; dashed line = onset of error (see text). Positions 1, 3 and 5 stay high under every constraint and correspond to the dead tokens discussed below, and highlighted by Fig. 5. On top of each curve, we indicate the token position it corresponds to, and the parent (level, position) that the parent constrained metric uses.

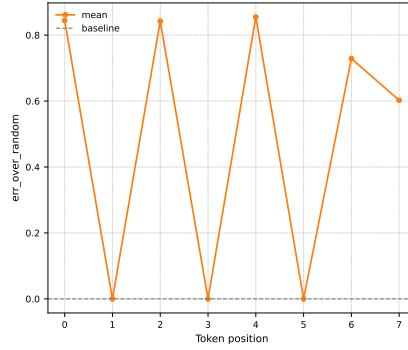


Figure 5: **Per-token zero-ablation after Block₁ (SAE₁).** Transformer without residual stream, RHM configuration $v = 16, L = 3, m = 16$. The post-block stream at one position u is set to zero (all other positions left intact) and the classification error, normalized by the random-chance baseline, is recorded. Positions 1, 3 and 5 (the dead tokens) have no effect when ablated.

From Figs. 4, 6, and 7 we can observe that the overall feature-latent association with level constraint works well for all three SAEs, since they globally reach close to 0 averaged entropies under that setting.

Dead tokens. In Fig. 4, we distinctly observe that token positions 1, 3 and 5 seem off, with high entropies under all constraints. We hypothesized that this indicated that the token positions of the transformer do not carry any relevant information between Block₁ and Block₂ for the downstream classification task. Indeed, to resolve the root of the RHM in a natural way, you can divide by s (2 in our setting) the different pieces of information needed at every layer, by completely resolving the parent latent. We checked this experimentally by performing an ablation analysis, zeroing out the output of each token position at a time after Block₁ and computing the new normalized error, and observed no impact for these “dead token” positions (Fig. 5).

Sparsity impact. As λ increases from small values, entropy curves tend to decrease monotonically before the onset of error, with the exception of the dead tokens in Fig. 4 described previously. The SAEs become more informative about the RHM’s latent variables as the dictionary is compressed. We notice in Fig. 6 and 7 that level-constrained entropy either reaches zero or attains its minimum very slightly before the onset of error, after which the loss of reconstruction quality degrades the

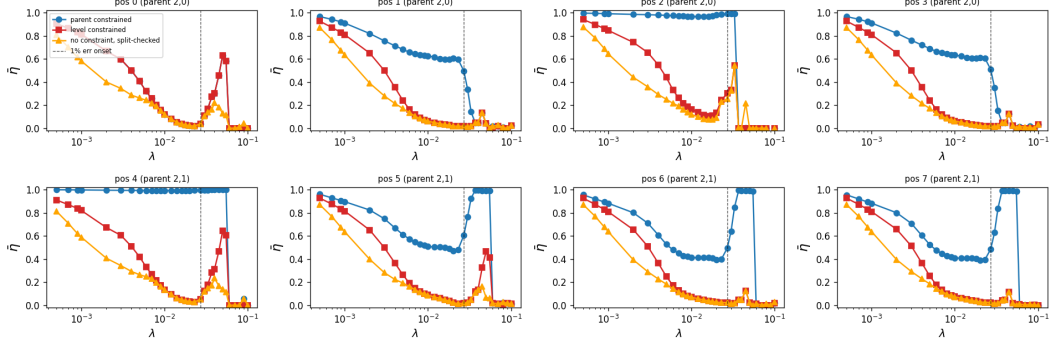


Figure 6: **Entropy curves for SAE₂, all token positions.** RHM parameters $v = 16, L = 3, m = 16$. Blue = parent-constrained, red = level-constrained, orange = unconstrained; dashed line = onset of error (see text). Level-constrained entropy reaches ≈ 0 before onset while parent-constrained stays above 0.4 (except for position 0): locality is too strong here.

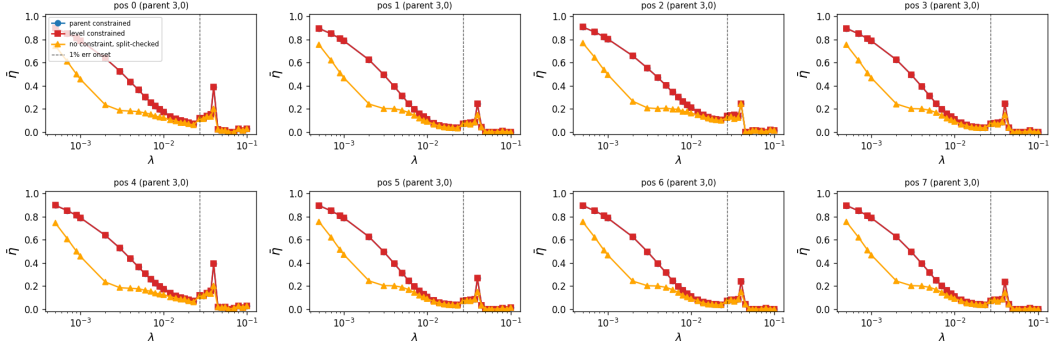
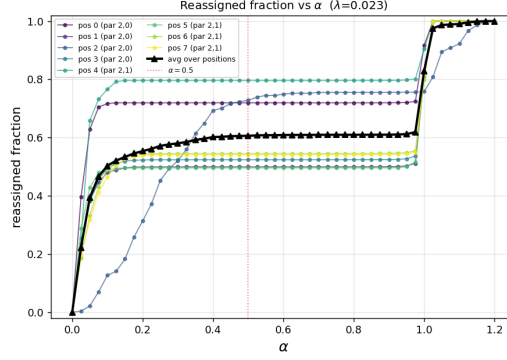


Figure 7: **Entropy curves for SAE₃, all token positions.** RHM parameters $v = 16, L = 3, m = 16$. Blue = parent-constrained, red = level-constrained, orange = unconstrained; dashed line = onset of error (see text). Parent- and level-constrained curves overlap trivially since the root has a single token position.

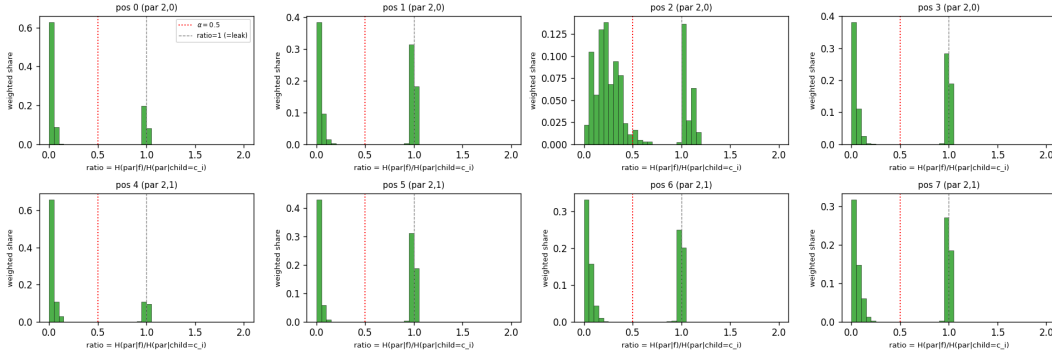
network’s computation. This further confirms the idea that SAEs of interest live in that region of λ values, in our case between 10^{-2} and $3 \cdot 10^{-2}$: sparse enough for features to be highly selective, but not so sparse as to break the model.

Delocalization phenomenon. The fact that parent-constrained labeling is too restrictive is clearly highlighted by Fig. 6. Indeed, we observe that while parent curves, except for token position 0, never drop below 0.4 weighted entropy value, level curves get very small when lambda increases. Clearly, the locality hypothesis is too strong here.

Feature absorption evidence. We focus on SAE₂ for this paragraph, though SAE₃ has very similar behavior. From Fig. 6, we see that the feature-latent value association is accurate in the level-constrained setting. However, when looking at Fig. 9, we observe that when we lift off the level constraint and examine λ values close to the onset of error, the share of features (weighted by their mean activation for that position) associated with level 1 latent values is often larger than, and always comparable to, the share of those associated with level 2 latent values. We checked for evidence of feature absorption using the protocol established in Sec. 4.2. Fig. 8a reveals a global plateau, and the corresponding entropy-ratio histograms (Fig. 8b) are directly bimodal, with a clean valley between absorbed and genuine features. The plateau is what we used to fix $\alpha = 0.5$ in the stable region, which we use for the reassignment of absorbed features to a higher-level latent; we report the new assigned level distribution in Fig. 10. In the latter figure, we now observe that near the onset of error, all features are assigned (or have been reassigned) to a matching level-2 parent.



(a) Reassigned fraction vs α , non-residual variant.



(b) Entropy-ratio distribution across token positions, non-residual variant.

Figure 8: **Reassignment threshold and the ratio distribution, SAE₂, non-residual variant.** RHM parameters $v = 16, L = 3, m = 16, \lambda = 0.023$. (a) The reassigned fraction shows a clear plateau, so the chosen α is insensitive within it. (b) The corresponding entropy-ratio histograms are bimodal, with a clean valley near $\alpha = 0.5$ separating absorbed features from genuine ones; the part of the distributions located around 1 is mainly due to the presence of genuine level 2 features (see the distribution curves of Fig. 9 and 10).

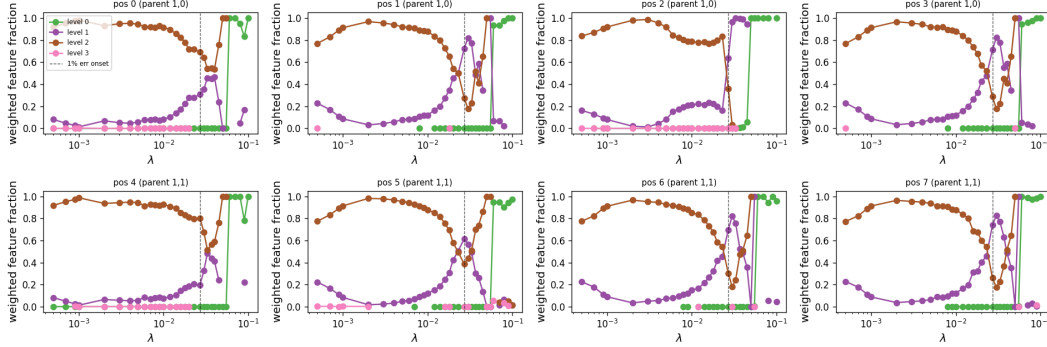


Figure 9: **Distribution of RHM level contribution for unconstrained labeling of SAE₂, before reassignment.** RHM parameters $v = 16, L = 3, m = 16$. Each panel is a token position, and curves track the fraction of features (weighted by their mean activation) assigned to each RHM level under unconstrained arg min labeling. Level 2 and level 1 fractions are still comparable close to the onset of error.

Experiments on other RHM parameters. We detail in App. A, Tab. 1 the other RHM architectures we trained Transformers and SAEs on, and the figures are available in App. D. For both other instances, we observe solid level-constrained feature-latent association close to the onset of error,

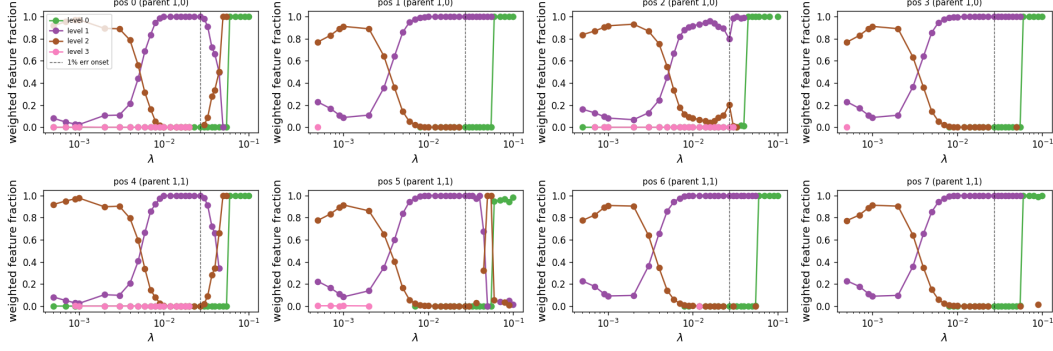


Figure 10: **Distribution of RHM level contribution for SAE₂, after reassignment.** RHM parameters $v = 16$, $L = 3$, $m = 16$. Same as Fig. 9 but after reassigning absorbed features to their parent level at $\alpha = 0.5$; the mass returns to level 2.

clean separation of genuine and absorbed features using the entropy ratio, and evidence of dead tokens between Block₁ and Block₂, though it isn't as pronounced for the $L = 3$ variant. While the $v = 8$, $L = 4$, $m = 8$ instance presents the same horizontal delocalization observed in this section (high parent-constrained curves, close to zero level-constrained ones), it isn't the case with our $v = 16$, $L = 3$, $m = 4$ one.

4.4 Experimental results on transformers with residual stream

While we recover some of the behavior observed for SAEs trained on transformers without a residual stream, the overall feature analysis is more complex for the residual variant.

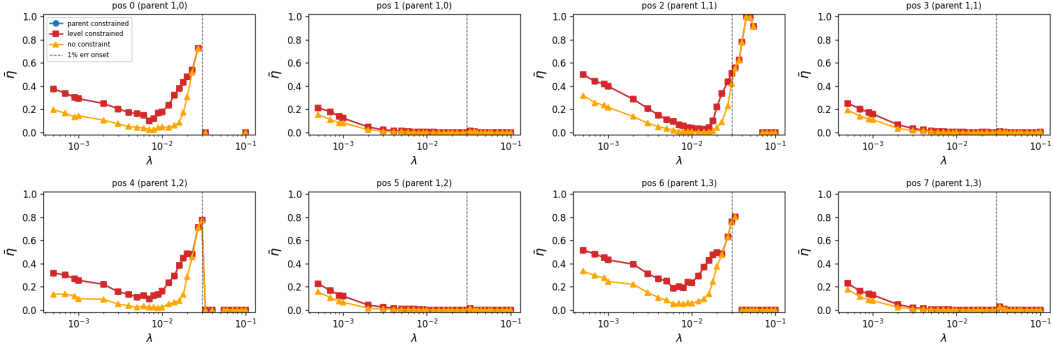


Figure 11: **Entropy curves for SAE₁, all token positions.** RHM parameters $v = 16$, $L = 3$, $m = 16$. Blue = parent-constrained (overlapping the red curve, locality assumption seems to hold), red = level-constrained, orange = unconstrained; dashed line = onset of error (see text). Positions 0, 2, 4, and 6 correspond to "dead tokens", as showcased by Fig. 12.

Dead tokens. From Fig. 11, we observe that SAE₁ on this variant showcases the same dead-token phenomenon. While positions 1, 3, 5 and 7 have level-constrained entropies reaching 0 well before the onset of error, it isn't the case for positions 0, 2, 4 and 6. The dead token hypothesis is further confirmed by the ablation analysis illustrated in Fig. 12.

Locality. For SAE₂, Fig. 13 displays that the parent and level-constrained curves overlap, meaning locality holds here: constraining to the matching level gives no improvement over constraining to the single parent latent. A natural reading is that the residual stream never discards the positional embedding, so each position keeps pointing to its own subtree across depth, as the non-residual variant does not.

Diverse features. More striking is that unconstrained labeling now clearly outperforms the level-constrained case for both SAE₂ and SAE₃ (Figs. 13, 14). Skip connections push low-level infor-

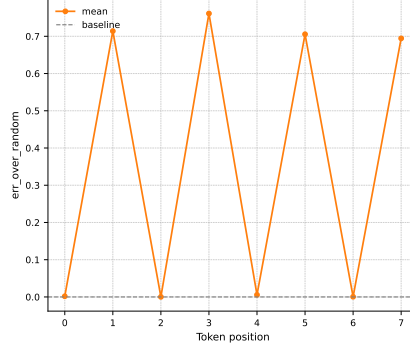


Figure 12: **Per-token zero-ablation after Block₁ (SAE₁).** Transformer with residual skip connections, RHM configuration $v = 16$, $L = 3$, $m = 16$. The post-block stream at one position u is set to zero (all other positions left intact) and the classification error, normalized by the random-chance baseline, is recorded. Positions 0, 2, 4, and 6 (the dead tokens) have no effect when ablated.

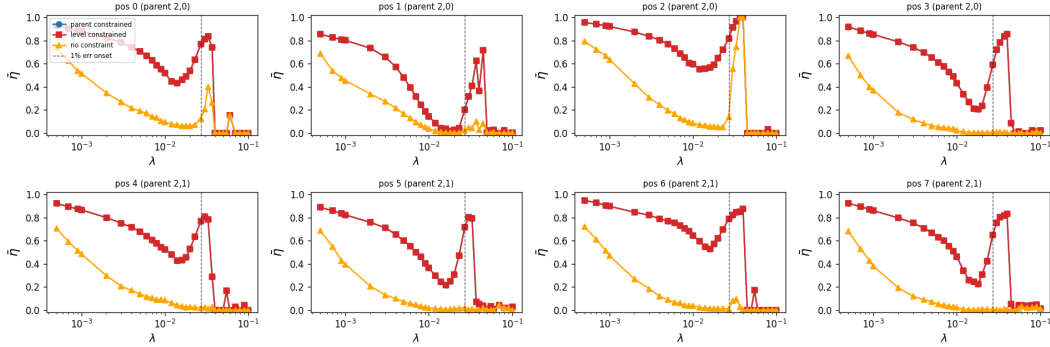


Figure 13: **Entropy curves for SAE₂, all token positions.** RHM parameters $v = 16$, $L = 3$, $m = 16$. Blue = parent-constrained, red = level-constrained, orange = unconstrained; dashed line = onset of error (see text). Parent and level-constrained curves still fully overlap, meaning locality holds.

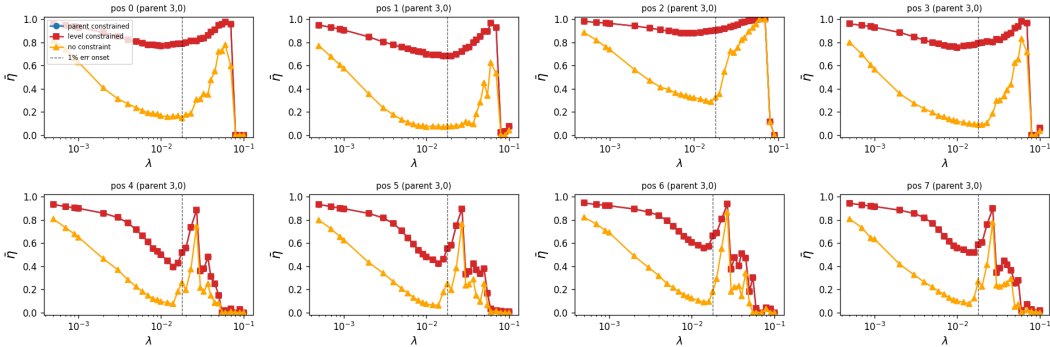
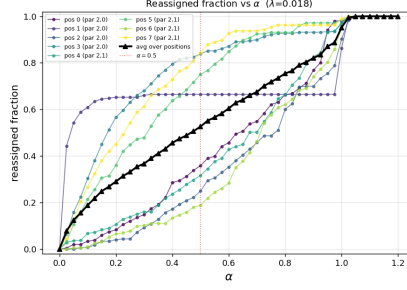
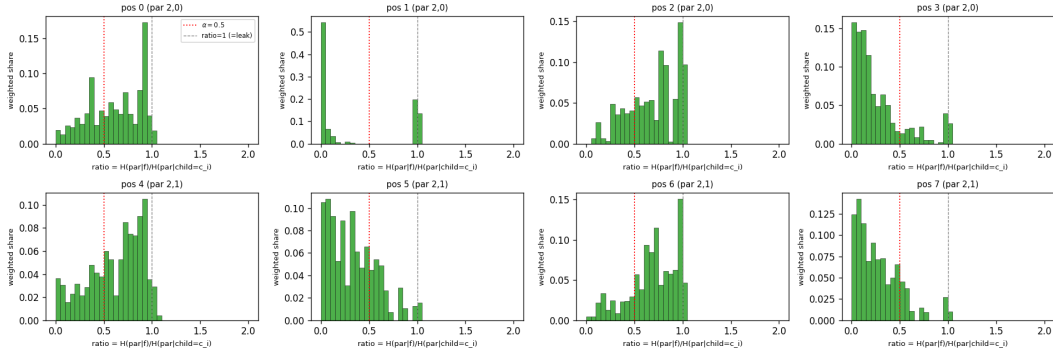


Figure 14: **Entropy curves for SAE₃, all token positions.** RHM parameters $v = 16$, $L = 3$, $m = 16$. Blue = parent-constrained, red = level-constrained, orange = unconstrained; dashed line = onset of error (see text). Parent and level-constrained curves overlap trivially since the root has a single token position.

mation into deeper layers, so genuine low-level features are expected. The reassignment test of Sec. 4.2 should then behave as in the non-residual case, with a bimodal ratio and a lower plateau in the reassigned fraction. However, Fig. 15a shows the reassigned fraction rising gradually with α , and the histograms (Fig. 15b) are unimodal at almost every position, with no valley, so no clear threshold separates genuine from absorbed features. Either the SAE fails to separate the two kinds of feature



(a) Reassigned fraction vs α , residual variant.



(b) Entropy-ratio distribution for all token positions, residual variant.

Figure 15: **Reassignment threshold and the ratio distribution, SAE₂, residual variant.** RHM parameters $v = 16, L = 3, m = 16, \lambda = 0.018$. (a) The reassigned fraction rises gradually with α , with no plateau. (b) The corresponding entropy-ratio histograms have weight all across $[0, 1]$ (position 1 excepted), so no threshold cleanly separates genuine features from absorbed ones, unlike the bimodal non-residual case.

in this setting, or the activation space is carved into features on a continuum between genuine and absorbed rather than in two groups.

SAE on the pooled representation after Block₃. We trained a SAE on the pooled representation

$$\tilde{x}^{(L)} = \frac{1}{s^L} \sum_{u=0}^{s^L-1} \text{LN} \left(x_u^{(L)} \right),$$

that precedes the final linear classifier. As showcased in Fig. 16, feature analysis of this SAE is much more straightforward. We obtain very low entropy before the onset of error under root constraint, which coincides with parent and level constraint here. Moreover, features assigned to lower-level latent values under unconstrained labeling are all "absorbed" features, which also specify the root value, as showcased both by the histogram and the level distribution after reassignment.

Experiments on other RHM parameters. Once again, we replicated the analysis on the remaining RHM configurations detailed in Tab. 1, with figures available in the App. D. For both RHM sets of parameters, SAE₁ and SAE_{pool} still exhibit very low parent- and level-constrained entropies near the onset of error, while the intermediate SAEs display higher level-constrained entropies. The entropy-ratio distributions are similarly more spread out, not always offering a clean threshold to separate absorbed features from genuine lower-level ones. The dead-token phenomenon between Block₁ and Block₂ is also consistently observed.

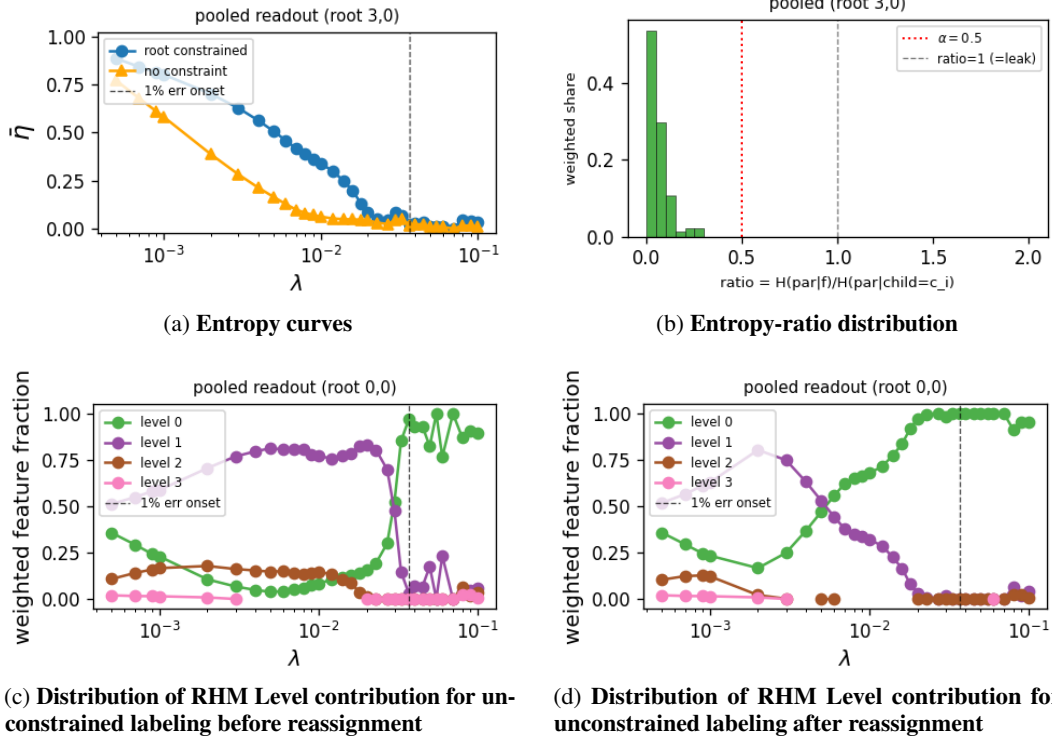


Figure 16: **Feature analysis curves for the SAE trained on the pooled representation after Block₃ of the residual variant.** RHM parameters $v = 16$, $L = 3$, $m = 16$. (a) The root-constrained entropy curves get close to 0 before the onset of error. (b) Computed at $\lambda = 0.027$, the entropy-ratio distribution is concentrated around 0: all features assigned to lower-level latent values under the unconstrained labeling scheme are also specific to the root value. (c) Before reassignment, not so far from the onset of error, there is still a substantial number of features assigned to level 2 latents. (d) After reassignment within that same region, all active features are labeled with a root value (and possibly a child latent value).

5 Circuit tracing

Feature analysis tells us what the SAE features represent, but not how the model combines them to classify the root. Circuit tracing is meant to recover that computation by building an *attribution graph* for a single input [12]. Its nodes are the features active on that input, and a directed edge between two features measures how much the upstream one contributes to the activation of the downstream one. Following the graph from the leaves up to the predicted root shows which features the model relies on and how information moves between them.

On a language model, features are identified either by a human or another language model. Moreover, language is complex, and there is no reference circuit to compare findings against. To obtain interpretable circuits, features have to be hand-picked and grouped by hand into large “supernodes”. In the case of the RHM, the ground truth circuit is known: we label our features against the known latents, and we can compare the circuits we obtain to the one that generated the data. However, this comparison is more subtle than it looks. The tree is the way the data is generated, not necessarily the way the transformer computes the root. While previous results show that networks trained on the RHM exploit its hierarchical, compositional structure, building synonym-invariant representations and relying on local correlations [14–16, 18], they don’t trace the step-by-step computation the trained model actually performs. Recovering the tree from our attribution graphs would therefore also tell us something about the transformer itself, not just about our tracing method, and a mismatch could go either way: the model computing the root differently, or our graph not being faithful to it.

5.1 Method

Prior work. Circuit tracing with interpretable sparse features has been developed mostly over the last few years [11–13]. Some of the most striking results come from Anthropic’s interpretability team, who build attribution graphs from Cross-Layer Transcoder (CLT) features. Similar in spirit to SAEs, CLTs are trained post hoc to replace the transformer’s feed-forward layers with a sparse set of features, each allowed to write into all downstream layers. We adapt this approach to our setting, using our per-block SAE features as the nodes of the graph.

A local replacement model. The construction rests on rewriting the transformer’s computation on a single datum in terms of features, while keeping it numerically exact. This is the *local replacement model* of [12], which we adapt from their cross-layer transcoders to our SAEs. On a fixed input we make two changes to the forward pass. First, at every post-block site we replace the stream by its SAE reconstruction and keep the leftover as an explicit error term,

$$x_p^{(k)} = \hat{x}_p^{(k)} + e_p^{(k)}, \quad \hat{x}_p^{(k)} = b_{\text{dec}} + \sum_i z_i(x_p^{(k)}) W_{\text{dec}}[:, i] \quad (9)$$

for every position $p \in \{0, \dots, s^L - 1\}$, so the active features $z_i > 0$ are the units we read and $e_p^{(k)}$ absorbs whatever the dictionary misses. Second, we freeze every input-dependent nonlinearity at the value it took on that datum: the LayerNorm scales, the attention softmax patterns, and the MLP ReLU masks. Once the error terms are kept and the nonlinearities frozen, the rewritten model reproduces the original activations and logits exactly. The construction is local in that it is built for this one datum, and on that datum it is exact: the incoming edges of each feature sum to its pre-activation, so the graph is an exact linear decomposition of the computation [12].

Linearized blocks. With the nonlinearities frozen, each block becomes an affine map of the stream. We write M_k for the linear part of Block $_k$, which carries $(x_0^{(k-1)}, \dots, x_{s^L-1}^{(k-1)})$ to $(x_0^{(k)}, \dots, x_{s^L-1}^{(k)})$, $M_k \in \mathbb{R}^{s^L d_{\text{model}} \times s^L d_{\text{model}}}$, acting on the whole stream at once, since attention mixes positions, and carrying the frozen attention patterns, the LayerNorm scalings, the residual path, and the linearized MLP (constructed explicitly in App.C). Here, our setup differs from [12]: their features replace the MLP, so feature-to-feature paths run only through attention and the residual stream, whereas our features sit on the residual stream and M_k is the whole block linearized, MLP included. We write $\delta_p \otimes v$ for the perturbation placing a vector v at position p and zero elsewhere, and $M_k(\cdot)[q, :]$ for its output read at position q .

Nodes. The graph has four kinds of nodes. *Feature nodes* (k, p, i) , one per feature active on the input. *Error nodes* (k, p) , one per layer and position, carrying $e_p^{(k)}$; they feed downstream features but receive no edges. *Embedding nodes* (p) , one per leaf, holding the token-plus-position embedding fed into the first block, are pure inputs. *Logit nodes* (c) , one per class, are pure sinks.

Edges. Following [12], an edge is a virtual weight: the source activation times how much one unit of the source drives the target feature’s pre-activation through the linearized model. From feature i at (k, p) to feature j at $(k+1, q)$,

$$w[(k+1, q, j) \leftarrow (k, p, i)] = \underbrace{z_i(x_p^{(k)})}_{\text{source activation}} \underbrace{W_{\text{enc}}[j, :] \cdot M_{k+1}(\delta_p \otimes W_{\text{dec}}[:, i])[q, :]}_{\text{one unit of source drives target}}. \quad (10)$$

The second factor pushes the source’s decoder direction through the linearized block and reads how much it raises the target’s pre-activation. The other edge types reuse the same expression with the source direction swapped: an error node sends $e_p^{(k)}$ in place of $z_i W_{\text{dec}}[:, i]$, and an embedding node sends its embedding through M_1 into the first SAE. At the top, the “next block” is the classifying head: we freeze the final LayerNorm LN_f , mean-pool, and read the classifier, so feature i at (L, p) reaches logit c with weight $\frac{1}{s^L} z_i(x_p^{(L)}) W_{\text{cls}}[c, :] \cdot \text{LN}_f(W_{\text{dec}}[:, i])$, the $1/s^L$ coming from the pooling.

Attribution graph and pruning. We stack the four kinds of nodes into one ordered list: embeddings first, then each block $k = 1, \dots, L$ by position with that position’s error node, and the logit nodes last. We read the graph off a single adjacency matrix A , where $A[t, s]$ is the signed weight of

the edge from source s to target t (Fig. 17). In this order, A is strictly triangular: edges only go embedding $\rightarrow$ block-1, block- $k \rightarrow$ block- $(k+1)$, and block- $L \rightarrow$ logit, so every edge runs from an earlier node to a later one. To rank nodes by influence rather than by a single edge weight, we use $\tilde{A}$, the unsigned A with each row divided by its total incoming weight, and form

$$B = \tilde{A} + \tilde{A}^2 + \dots + \tilde{A}^{L+1}, \quad (11)$$

so that $B[t, s]$ sums the influence carried from s to t along paths of every length. Since A is strictly triangular, the longest path is the $L + 1$ hops from an embedding to a logit, $\tilde{A}$ is nilpotent, and the sum is exact. Pruning then proceeds in two stages, controlled by a node threshold τ_{node} and an edge threshold τ_{edge} :

1. *Nodes.* Score each feature and error node by its influence on the logits, $\text{score}(v) = \sum_{t \in \text{logits}} B[t, v] \pi(t)$, where $\pi(t)$ is the softmax probability of class t (very concentrated on the true class). Keep the smallest set of feature nodes whose cumulative score reaches a fraction τ_{node} of the total.
2. *Edges.* Recompute A , $\tilde{A}$, B on the surviving subgraph, score each edge by $\tilde{A}[t, s] \text{score}(t)$, and keep the smallest set reaching a fraction τ_{edge} of the total.

Embedding and logit nodes are never removed; only feature and error nodes, and edges are pruned. The two thresholds τ_{node} and τ_{edge} are the only knobs of the procedure, and set how aggressively the graph is sparsified.

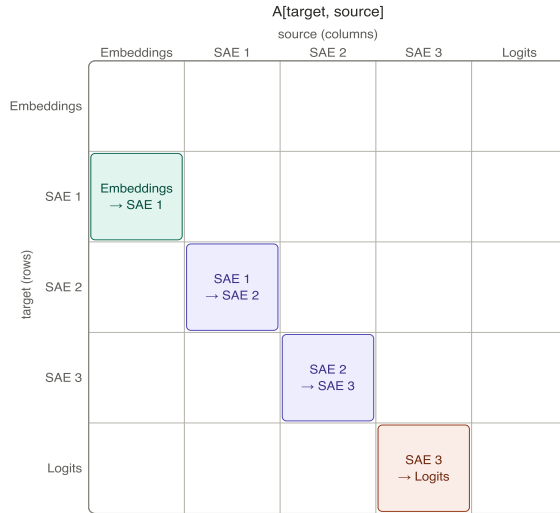
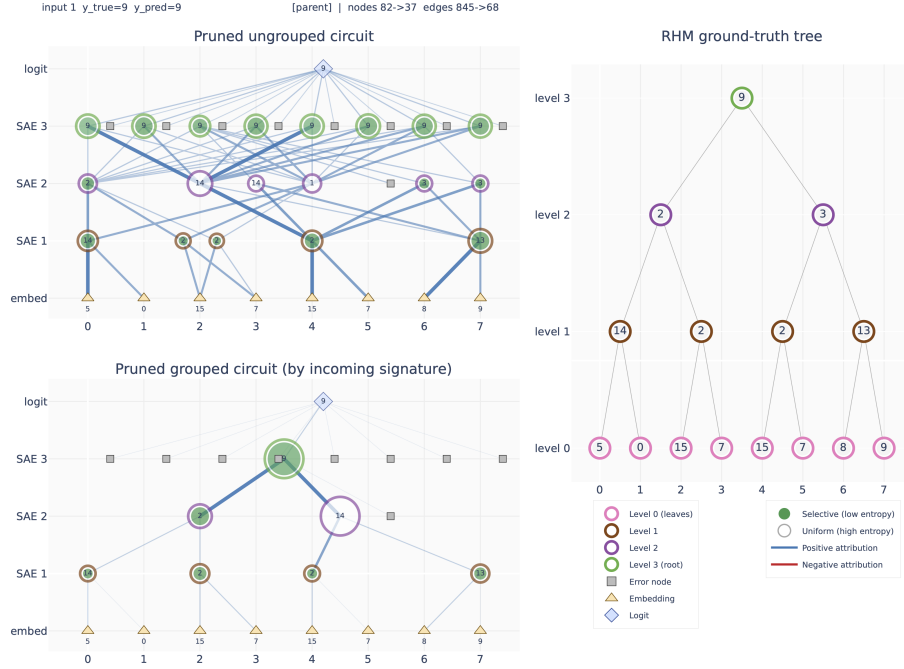


Figure 17: **Block structure of the adjacency matrix A .** RHM of depth $L = 3$. Nodes are ordered by depth: embeddings, then each block $1, \dots, L$ (the active feature nodes and the per-position error nodes at that layer), then the logits. Edges only connect adjacent groups, so the nonzero blocks sit strictly below the diagonal and A is nilpotent ($A^{L+2} = 0$); this is what makes the influence sum B in Sec. 5.1 finite and exact. Error nodes are sources only and logit nodes sinks only.

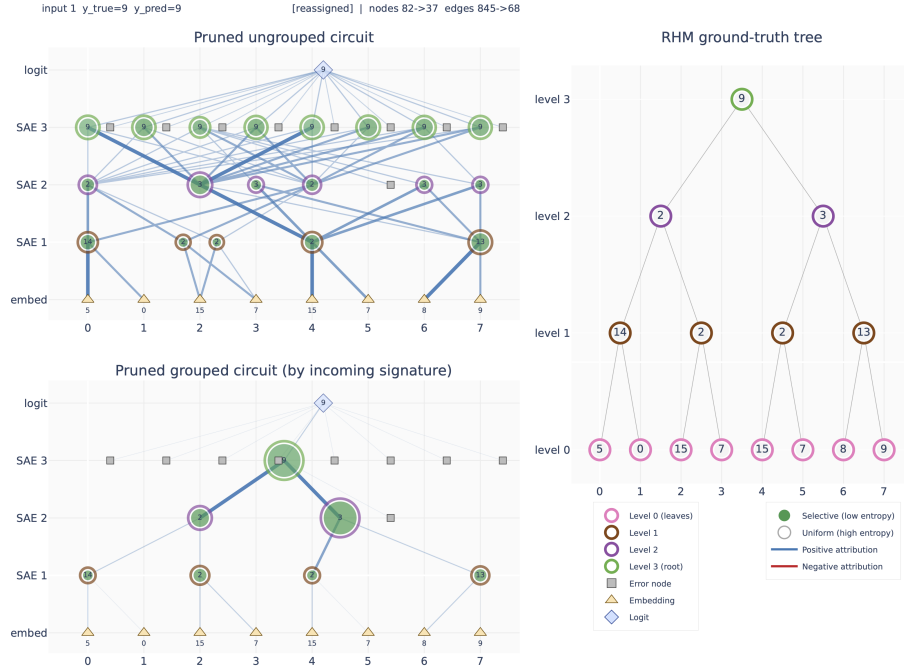
Grouping. We present two kinds of pruned circuit, grouped and ungrouped. We introduce grouping as a post-pruning step meant to make the circuits more readable. Even after pruning, several features encoding the same latent value remain active, at different positions or at the same one. For language models, circuits are simplified by hand-picking and merging features from human judgment [12]. For the RHM, we instead group features systematically by their *incoming signature*: the set of previous-layer features that still feed a feature after pruning. For a surviving feature node $v = (k, p, i)$, the incoming signature is

$$\text{sig}(v) = \{u \in \mathcal{F}_{k-1} : A^*[v, u] \neq 0\}, \quad (12)$$

where $\mathcal{F}_{k-1}$ are the surviving SAE features at layer $k - 1$ and A^* the pruned adjacency. Within a layer we merge all features sharing the same signature, $v \sim v' \iff \text{sig}(v) = \text{sig}(v')$. The procedure runs bottom-up, with the embedding nodes as the level-0 features.



(a) **Circuit tracing example with parent constrained labeling.** RHM parameters $v = 16$, $L = 3$, $m = 16$ for a transformer without residual stream. $\lambda = \{0.023, 0.02, 0.02\}$ for SAEs 1, 2 and 3. $\tau_{\text{node}} = 0.8$, $\tau_{\text{edge}} = 0.9$.



(b) **Circuit tracing example with reassigned labeling.** RHM parameters $v = 16$, $L = 3$, $m = 16$ for a transformer without residual stream. $\lambda = \{0.023, 0.02, 0.02\}$ for SAEs 1, 2 and 3. $\tau_{\text{node}} = 0.8$, $\tau_{\text{edge}} = 0.9$. $\alpha = 0.5$ for the reassignment scheme.

Figure 18: Comparison of circuit-tracing labeling schemes.

5.2 Results

We illustrate circuit tracing on the architecture $v = 16$, $L = 3$, $m = 16$. After grouping and under unconstrained but reassigned labeling, the pruned attribution graphs can recover the RHM topology on individual inputs for transformers without residual connections (see Fig. 18), but often contain off-level features when residual connections are present (see Fig. 19). In the figures, feature color encodes the entropy η , from white at $\eta = 1$ to green at $\eta = 0$ for fully selective features; colored rings mark the assigned RHM level, and edge color distinguishes positive from negative edges, with feature and edge sizes proportional to their values. For each transformer, we fix λ slightly before the onset of error, where entropy metrics are minimized, and prune with node and edge thresholds swept over $\{0.75, \dots, 0.95\}$ and $\{0.9, \dots, 0.99\}$, using $\alpha = 0.5$ for reassignment. The method also produces interactive HTML circuits, five per configuration and variant of Tab. 1, which add per-feature details such as activation value, assigned probability, and constituents, and are available at [RHM-circuit-examples](#) [22].

Grouping recovers the RHM topology. Fig. 18 showcases circuits for one input through a non-residual transformer, changing the labeling scheme from parent-constrained to unconstrained with reassignment for feature absorption. After grouping, the recovered graph matches the datum’s generative tree in a directly visible way. Under parent-constrained labeling (Fig. 18a), several SAE_2 features appear mislabeled: they are white, with near-random entropy over their parent latent, and encode a value absent from the ground-truth tree. Switching to unconstrained with reassigning labeling (Fig. 18b) removes this, and every feature in the graph then identifies its latent value correctly and is highly selective.

Delocalized storage. These mislabeled features are informative about the model itself. The features labeled 14 at positions 2 and 3 are grouped with the selective features labeled 3 at positions 6 and 7, and all four carry the same SAE feature index i . Information about $h_1^{(2)}$ is therefore stored not only beneath it, at positions 6 and 7, but also at positions 2 and 3, where parent-constrained labeling cannot find it. This is the same delocalization seen in the feature analysis of Sec. 4.3, now visible in a single circuit. Such findings, while performing early circuit tracing, motivated the development of different types of constraints for feature labeling.

Residual transformers. Fig. 19 shows a circuit for the same configuration with residual connections. The overall structure remains consistent with the ground truth, but some features keep off-level latent labels even under reassigned labeling, consistent with the labeling ambiguity reported for the residual variant in Sec. 4.4.

Dead tokens. The dead-token phenomenon of Sec. 4.2 is consistent with these circuits. The ablation in Fig. 5 marked positions 1, 3 and 5 as uninformative between Block_1 and Block_2 , and in Figs. 18a and 18b no important information flows through those positions between the two blocks. The same holds for the surviving SAE_1 features in Fig. 19 against the residual dead tokens of Fig. 12, with the exception of position 4.

Other RHM architectures. We also traced circuits for the two other configurations of Tab. 1, available at [RHM-circuit-examples](#). For $v = 16$, $L = 3$, $m = 4$, with and without residual connections, the grouped circuits often directly coincide with the ground-truth RHM tree, and the residual case again keeps some off-level features. However, for both variants, the pruned circuits often have holes: some ground-truth latent values keep no feature, even at high thresholds. This may indicate that the model exploits the extra information carried by individual latent or leaf values, which is more informative at the low m of this configuration. For $v = 8$, $L = 4$, $m = 8$ the circuits are denser, partly because there are now $s^L = 16$ positions. The residual variant keeps more off-level features, and grouping tidies the graphs, but not as well as for $L = 3$. The dead-token pattern persists: usually only one position in every pair keeps SAE_1 features after pruning.

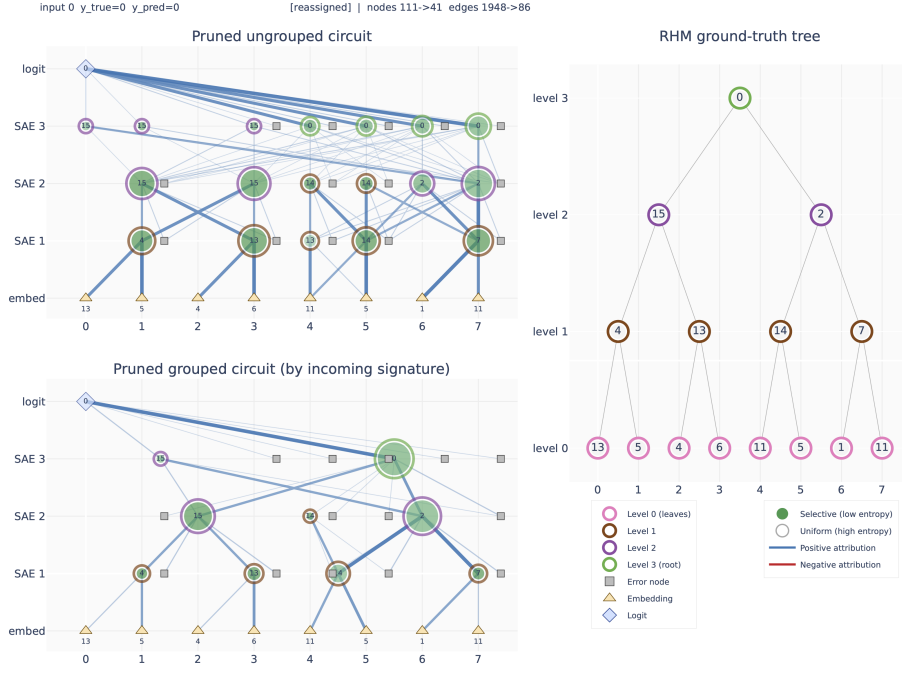


Figure 19: **Circuit tracing example with reassigned labeling.** RHM parameters $v = 16$, $L = 3$, $m = 16$ for a transformer with residual skip connections. $\lambda = \{0.023, 0.02, 0.02\}$ for SAEs 1, 2 and 3. $\tau_{\text{node}} = 0.9$, $\tau_{\text{edge}} = 0.98$. $\alpha = 0.5$ for the reassignment scheme.

6 Conclusion

We moved sparse autoencoders and circuit tracing from generative language models to classification on the Random Hierarchy Model. In doing so, we learned both about the interpretability method itself and what it revealed about how transformers mechanistically infer the root. Close to the onset of error in the sparsity sweep, several indicators agree on the dictionaries of interest: the level-constrained conditional entropy approaches zero, the mean number of active features per position falls to order one, and the classification error has not yet left its floor. The onset of error also gives a task-grounded way to severely restrict the search for a valid sparsity coefficient. Due to the hierarchy of our data, we encountered a reported behavior of SAEs: feature absorption creating even more specific features. We can rigorously identify these over-specific features, but only partly in the presence of skip connections. This is worth investigating further. Without these skip connections, the feature analysis is straightforward: active features cleanly identify their corresponding RHM-level latents for the most part, and we can rigorously distinguish absorbed lower-level features thanks to the entropy ratio we defined. When transformers do have skip connections, SAEs encode more lower-level information as expected, but our metric doesn't distinguish genuine from absorbed features as cleanly. Two findings concern how SAEs accurately reveal how information flows through the model. The first was the horizontal delocalization phenomenon in non-residual networks, flagged by the entropy metrics and confirmed by the circuit-tracing examples. Secondly, the phenomenon of dead tokens, specifically due to the RHM topology, was identified by both entropy metrics and ablation experiments. Finally, we showcased examples in which our circuit-tracing method and automatic grouping algorithm recover the correct generative tree for $L = 3$ depth.

The RHM allowed us to systematically assess SAE performance and develop a grounded approach to setting sparsity, maximizing feature selectivity near the onset of error. We proposed a way to quantitatively address absorption, a key concern when dealing with hierarchical concepts inherent in natural data. This last method, however, since it uses the particular production rules, was specific to the RHM, and it would be necessary to extend it to fit the language setting as well. The grouping procedure for circuit tracing relies on the fact that our generative procedure is tree-like, and could be

insightful for language circuits as well, since natural language is commonly modeled as compositional and tree-structured. Overall, the dead tokens and the delocalization are concrete examples of SAEs and circuit tracing revealing how the model mechanistically deals with the RHM’s structure.

Limitations. These results are bounded by the numerous choices the methods require, including hidden layer dimension, edge and pruning thresholds, for which we have yet to come up with principled ways to set them. The use of SAEs rests on the linear representation and superposition hypotheses [6, 23], under which concepts are independent linear directions in activation space. Recent work questions this, finding that some concepts lie on low-dimensional manifolds rather than single directions [24, 25], and [26] argue that an SAE tends to spread such structure across many features, so it is hard to read off any single one.

Future work. To improve our current analysis, we could develop a way to quantitatively measure how much circuits match with the ground truth. An idea would be to compute the share of edges that are "RHM-coherent", meaning they link parent and child features of the ground truth. A rigorous version of this score is nontrivial and depends on the pruning thresholds, sparsity coefficients, and labeling schemes. The next direction would be to move from root classification to next-token prediction, in which the RHM provides a rigorous setting for developing a strong theory of how LLMs function. There, the optimal predictor is belief propagation on the tree [15]. It gives an explicit target: directed messages on a known depth schedule, against which circuit tracing can be checked. Two things are then worth exploring. First, whether the task’s directionality, since each prediction is read from the last token, sharpens the layer-to-level correspondence that pooling blurs under classification. Second, a labeling adapted to next-token prediction that keeps, for each feature, the few latents on which its entropy is lowest, and their corresponding value.

7 Acknowledgments

First of all, I would like to express my gratitude to my supervisor, Matthieu Wyart, for making this internship possible and for his insightful guidance throughout. I also want to warmly thank Alessandro Favero for his precious help and overview of the project.

I want to give very special thanks to all the members of the group for their crucial help and general advice, especially Daniel Korchinski, Laurent Bugnard, Francesco D’Amico, Corinna Wegner, Jack Parley, Corinne Weibel, and Francesco Cagnetta.

Additionally, I would like to thank the Swiss European Mobility Program for sponsoring my internship, and SCITAS for offering the computing resources necessary for the experiments.

Finally, I want to thank the ENS Physics Department for their support, and Giulio Biroli for helping me find this internship.

References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners, July 2020. arXiv:2005.14165 [cs.CL].
- [2] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent Abilities of Large Language Models, October 2022. arXiv:2206.07682 [cs.CL].
- [3] Leonard Bereska and Efstratios Gavves. Mechanistic Interpretability for AI Safety – A Review, August 2024. arXiv:2404.14082 [cs.AI].
- [4] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A Mathematical Framework for Transformer Circuits, December 2021.
- [5] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context Learning and Induction Heads, March 2022.
- [6] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy Models of Superposition, September 2022. arXiv:2209.10652 [cs.LG].
- [7] Lee Sharkey, Dan Braun, and beren. Taking features out of superposition with sparse autoencoders — AI Alignment Forum. December 2022.
- [8] Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, and Lee Sharkey. Sparse Autoencoders Find Highly Interpretable Features in Language Models. October 2023.
- [9] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas L Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. *Transformer Circuits Thread*, 2023, 2023.
- [10] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Natalie, Amelie Ley, Nicholas Joseph, Shauna Smith, Nicholas MacEachern, Alex Tamkin, Shanda Udell, Robert Lasenby, Izzy Kingsley, Catherine Olsson, Tristan Hume, Shan Carter, Christopher Olah, and Joshua Batson. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024, 2024.
- [11] Samuel Marks, Can Rager, Eric Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *International Conference on Learning Representations*, volume 2025, pages 23888–23923, 2025.
- [12] Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman,

- Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. Circuit Tracing: Revealing Computational Graphs in Language Models, March 2025. Published: Transformer Circuits Thread.
- [13] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermy, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. On the Biology of a Large Language Model, March 2025. Published: Transformer Circuits Thread.
 - [14] Francesco Cagnetta, Leonardo Petrini, Umberto M. Tomasini, Alessandro Favero, and Matthieu Wyart. How Deep Neural Networks Learn Compositional Data: The Random Hierarchy Model. *Physical Review X*, 14(3):031001, July 2024. arXiv:2307.02129 [cs.LG].
 - [15] Francesco Cagnetta and Matthieu Wyart. Towards a theory of how the structure of language is acquired by deep neural networks, May 2024.
 - [16] Antonio Sclocchi, Alessandro Favero, and Matthieu Wyart. A phase transition in diffusion models reveals the hierarchical nature of data. *Proceedings of the National Academy of Sciences*, 122(1):e2408799121, January 2025.
 - [17] Grzegorz Rozenberg and Arto Salomaa. *Handbook of Formal Languages*. January 1997.
 - [18] Jack T. Parley, Francesco Cagnetta, and Matthieu Wyart. Deep networks learn to parse uniform-depth context-free languages from local statistics, 2026. _eprint: 2602.06065.
 - [19] Robert Tibshirani. Regression Shrinkage and Selection Via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, January 1996.
 - [20] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *International Conference on Learning Representations*, volume 2025, pages 26721–26754, 2025.
 - [21] David Chanin, James Wilken-Smith, Tomáš Dulka, Hardik Bhatnagar, Satvik Golechha, and Joseph Bloom. A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders. 2025.
 - [22] Tigrane Ponsin. Circuit tracing library, <https://tigraneponsin.github.io/rhm-circuit-examples/>.
 - [23] Kiho Park, Yo Joong Choe, and Victor Veitch. The Linear Representation Hypothesis and the Geometry of Large Language Models, July 2024. arXiv:2311.03658 [cs.CL].
 - [24] Joshua Engels, Eric J. Michaud, Isaac Liao, Wes Gurnee, and Max Tegmark. Not All Language Model Features Are One-Dimensionally Linear, February 2025. arXiv:2405.14860 [cs.LG].
 - [25] Dhruva Karkada, Daniel J. Korchinski, Andres Nava, Matthieu Wyart, and Yasaman Bahri. Symmetry in language statistics shapes the geometry of model representations, February 2026. arXiv:2602.15029 [cs.LG].
 - [26] Usha Bhalla, Thomas Fel, Can Rager, Sheridan Feucht, Tal Haklay, Daniel Wurgaft, Siddharth Boppana, Matthew Kowal, Vasudev Shyam, Jack Merullo, Atticus Geiger, and Ekdeep Singh Lubana. Do Sparse Autoencoders Capture Concept Manifolds?, April 2026. arXiv:2604.28119 [cs.LG].

A Experimental details

A.1 RHM instances

Table 1: RHM instances ($s = 2$ throughout). P is the transformer training-set size, $P^* \sim v m^L$ the sample-complexity scale of [14], $P_{\max}$ the total number of distinct data.

v	L	m	input length s^L	$P_{\max}$	P	P^*
16	3	4	8	2.6×10^5	12,160	1,024
16	3	16	8	4.3×10^9	1,048,576	65,536
8	4	8	16	2.8×10^{14}	524,288	32,768

A.2 Transformer

Parametrization and Initialization. Every linear map in the attention and MLP sublayers is bias-free and is scaled in the forward pass by the inverse square root of its fan-in: the query, key, value, and output projections by $d_{\text{model}}^{-1/2}$, the two MLP layers by $d_{\text{model}}^{-1/2}$ and $(4 d_{\text{model}})^{-1/2}$. Attention uses 8 heads (so head dimension $d_{\text{model}}/8$). Blocks are pre-LayerNorm, $x \leftarrow x + \text{Dropout}(\text{Attn}(\text{LN}_1(x)))$ then $x \leftarrow x + \text{Dropout}(\text{MLP}(\text{LN}_2(x)))$; the no-residual variants drop both additive skips. The pooling head is a uniform mean over the s^L post-LayerNorm positions. The linear classifier has bias, and the weights are initialized as $\mathcal{U}(-\frac{1}{\sqrt{d_{\text{model}}}}, \frac{1}{\sqrt{d_{\text{model}}}})$. All attention and MLP weights are initialized i.i.d. standard normal $\mathcal{N}(0, 1)$; LayerNorm starts at unit gain and zero bias, and the final linear classifier uses the default PyTorch initialization.

Hyperparameters. AdamW at constant learning rate 10^{-3} , weight decay 10^{-4} , dropout 0.1, batch size between 16 and 1,280 depending on P , at most 20,000 epochs; checkpoints on a log-spaced step grid, test sets of 32,768 held-out samples. Seeds for rule sampling, data sampling, and initialization are drawn at random per run and recorded with each checkpoint; checkpoints include the production rules, so every downstream stage regenerates the exact same grammar.

A.3 Sparse autoencoders

Initialization. Decoder columns are random directions of fixed norm 0.1, the encoder is the decoder transpose, and all biases are zero.

Normalization. Activations are rescaled per layer to expected norm $\sqrt{d_{\text{model}}}$ before SAE training, making λ comparable across different SAE hidden dimensions. While we don't vary it in this report, it could prove useful for future work. The per-layer rescaling is a single scalar $\kappa = \sqrt{d_{\text{model}}} / \mathbb{E}\|x\|_2$, estimated on the SAE training set; κ is stored with the checkpoint and divided back out in all downstream analyses. We don't include it in the circuit-tracing equations for simplicity and clarity purposes.

Optimization. AdamW with 0 weight decay, learning rate 10^{-4} , 2^{17} gradient steps. Each step draws a batch of 128 RHM samples through the frozen transformer, yielding $128 \times s^L$ SAE training vectors; the SAE training set contains the same number P of RHM samples as the corresponding transformer's training set. The learning rate and batch size were selected by an Optuna search (learning rate log-uniform in $[10^{-6}, 10^{-2}]$, batch size in $\{64, 128, 256, 512, 1024\}$, $\lambda = 10^{-2}$ fixed, objective: final held-out SAE loss). SAE training data is generated from the same production rules as the transformer's, with a disjoint sampling seed; evaluations use a third sample with a seed disjoint from both. SAEs are trained against the lowest-test-loss transformer checkpoint; this choice is recorded and enforced in all downstream stages.

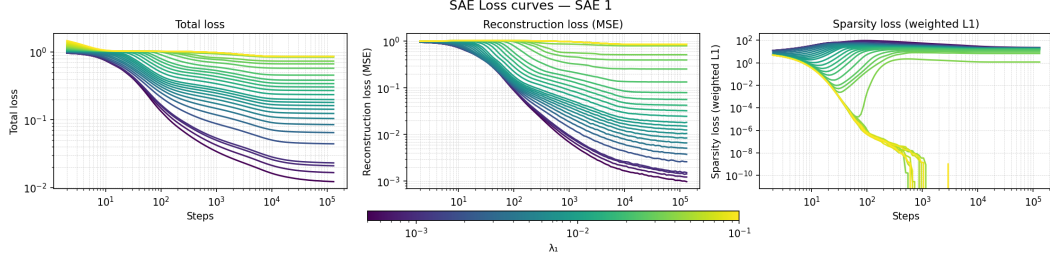


Figure 20: **Example of loss curves for an SAE sweep.** Loss curves on the test set for SAE_1 on a transformer trained for an RHM with $v = 16$, $L = 3$, $m = 16$. After a certain λ threshold, the sparsity loss goes to 0, and the SAE fits the mean activation vector with its biases.

B Locality assumption

This appendix expands the efficiency argument behind the localization assumption that constrained our first feature-analysis pipeline (Sec.4): that the post-block stream stores each latent at the positions lying beneath it in the tree.

The assumption. Under the layer-level correspondence, block k composes its input by one level, so that the stream $x^{(k)}$ resolves the level- k latents through the grammar’s inverse maps,

$$h_j^{(k)} = g_k \left(h_{js}^{(k-1)}, \dots, h_{(j+1)s-1}^{(k-1)} \right).$$

Localization adds a claim about *where* each latent is stored: $x_u^{(k)}$ carries $h_{\lfloor u/s^k \rfloor}^{(k)}$, the level- k ancestor of leaf u . Equivalently, the s^k positions of a subtree all hold that subtree’s level- k latent and no other. The argument for it reads attention literally, as the transport of content from one position to another: to evaluate g_k at a position, that position must hold all s children it composes, and must import by attention any it does not already carry.

Why locality is cheap. Take a level-1 latent with $s = 2$. The node $h_0^{(1)} = g_1(h_0^{(0)}, h_1^{(0)})$ depends only on leaves 0 and 1. Forming it at position 0 requires that position to hold both leaf embeddings; since it already carries leaf 0, a single attention interaction (0 reading from 1) suffices, and symmetrically at position 1. Storing the same kind of latent outside its subtree would be “more expensive”: to place $h_1^{(1)} = g_1(h_2^{(0)}, h_3^{(0)})$ at position 0, which holds neither leaf 2 nor leaf 3, position 0 must read from both 2 and 3 and use two interactions rather than one. In general a parent co-located with one of its s children is formed with $s - 1$ imports, a parent at a foreign position with s . Applied recursively, the same holds at every depth. This is a minimal-mechanism argument, not a constraint the trained model must satisfy. In fact, we see that it does not hold systematically, which is why we extended the search region of latent variables for feature analysis.

C The linearized block as a matrix

This appendix spells out the matrix M_k of Sec.5.1, which on the traced input maps $x^{(k-1)}$ to $x^{(k)}$, once its input-dependent nonlinearities are frozen. We work on a single datum, with $N = s^L$ positions and $d = d_{\text{model}}$; the stream is the stacked vector in $\mathbb{R}^{Nd}$, and a perturbation v has the same shape. Three quantities are read off the forward pass and held fixed: the LayerNorm denominators, the attention softmax patterns, and the MLP ReLU masks. With these frozen, every sub-operation of the block is linear.

LayerNorm. At position p , $\text{LN}(x_p) = \gamma \odot (x_p - \bar{x}_p \mathbf{1}) / \sigma_p + \beta$, with mean $\bar{x}_p$ and standard deviation σ_p over the d coordinates. Freezing σ_p , the linear part on a perturbation is

$$L_p = \frac{1}{\sigma_p} \text{diag}(\gamma) \left(I_d - \frac{1}{d} \mathbf{1} \mathbf{1}^\top \right) \in \mathbb{R}^{d \times d} \quad (13)$$

(β is constant and drops out of edge weights). LayerNorm acts position-wise, so the full operator is block-diagonal, $L = \text{blkdiag}(L_0, \dots, L_{N-1})$. The two LayerNorms of a block give $L^{(1)}$ and $L^{(2)}$, each with its own γ and frozen σ_p .

Attention. With the post-softmax pattern $A_h[i, j]$ of head h frozen, the output at position i is

$$\text{Attn}(u)_i = \sum_j \left(\sum_h A_h[i, j] O_h \right) u_j, \quad O_h = W_{O,h} W_{V,h} \in \mathbb{R}^{d \times d}, \quad (14)$$

linear in u . As one matrix it has $N \times N$ blocks of size $d \times d$, with block (i, j) equal to $T[i, j] = \sum_h A_h[i, j] O_h$. This is the only part that mixes positions, which is why M_k isn't block-diagonal.

MLP. With the ReLU mask $m_p = \mathbf{1}[W_1 z_p + b_1 > 0]$ frozen, ReLU becomes multiplication by m_p , and the linear part at position p is

$$G_p = W_2 \text{diag}(m_p) W_1 \in \mathbb{R}^{d \times d}, \quad (15)$$

block-diagonal across positions, $G = \text{blkdiag}(G_0, \dots, G_{N-1})$.

Composition. A pre-LayerNorm block computes $y = x + \text{Attn}(\text{LN}_1(x))$ then $\text{out} = y + \text{MLP}(\text{LN}_2(y))$. Writing the two branch operators $A_{\text{mat}} = T L^{(1)}$ and $G_{\text{mat}} = G L^{(2)}$, the linearized block is

$$M_k = (I + G_{\text{mat}})(I + A_{\text{mat}}) \text{ (with residual)}, \quad M_k = G_{\text{mat}} A_{\text{mat}} \text{ (no residual)}, \quad (16)$$

the no-residual variant dropping the two skip connections. The top map $x^{(L)} \rightarrow \text{logits}$ is linear the same way: freeze LN_f , mean-pool ($\frac{1}{s^L} \sum_p$), and apply the classifier.

Cost. M_k is $Nd \times Nd$ with $Nd = s^L d_{\text{model}}$. For $s = 2$, $L = 3$ this is 4096×4096 (≈ 67 MB in float32), trivially formed by applying the affine map to each basis vector. For $s = 2$, $L = 4$ it is 8192×8192 ; above this size we never form M_k explicitly and instead evaluate $v \mapsto M_k v$ (or its transpose) on demand, one source or target at a time.

What the linearization keeps and drops. M_k is the block's Jacobian with stop-gradients on the three frozen quantities. On the traced input it reproduces the true block output, so the local replacement model matches the logits and each feature's pre-activation is exactly the sum of its incoming edges. The graph represents information routed through frozen attention (the OV circuit) and the MLP, but not the influence a feature exerts by changing the attention pattern (the QK circuit), the LayerNorm denominator, or which ReLUs are active. This is the same choice made by [12].

D Feature analysis of the other RHM instances

This appendix collects the feature-analysis figures for the two other RHM instances of Tab. 1 (the instance studied in the main text is $v = 16$, $L = 3$, $m = 16$). The conventions follow the main text: blue is parent-constrained, red is level-constrained and orange is unconstrained labeling, and the dashed vertical line marks the 1% error onset. In the Entropy ratio histograms, the red dotted line is the reassignment threshold $\alpha = 0.5$, and the grey dashed line marks ratio 1 (leak). The ablation panels show the post-Block₁ per-token zero-ablation error normalized by the random baseline.

D.1 $v = 16$, $L = 3$, $m = 4$, no residual stream

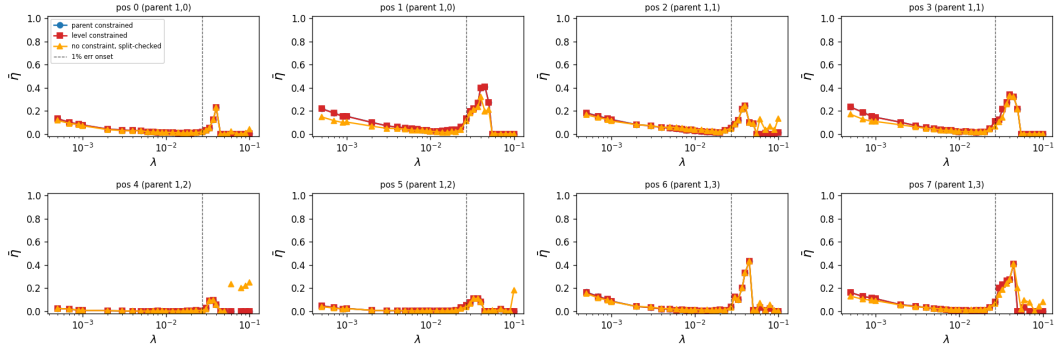


Figure 21: **Entropy curves for SAE₁, all token positions.** RHM parameters $v = 16, L = 3, m = 4$, no residual stream. Parent and level curves overlap, so locality holds, and every position falls to near zero before the onset.

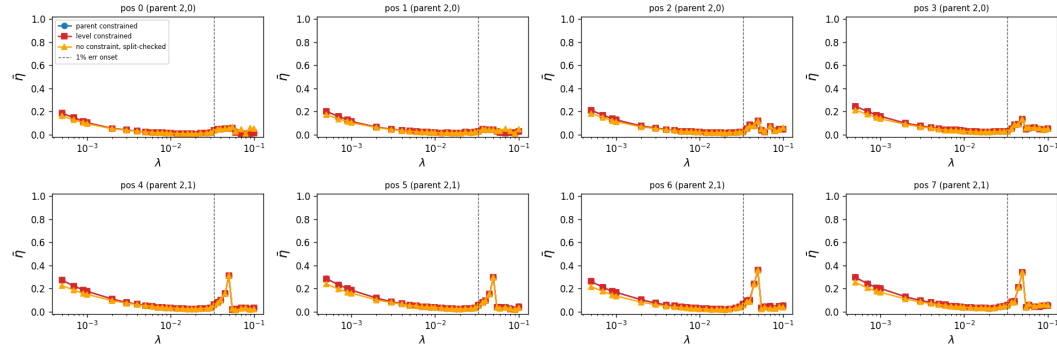


Figure 22: **Entropy curves for SAE₂, all token positions.** Same configuration. Parent and level curves overlap and reach almost zero just before the onset, with the unconstrained curve barely lower. Locality holds, so no delocalization appears at this intermediate layer, contrary to the $m = 16$ non-residual case.

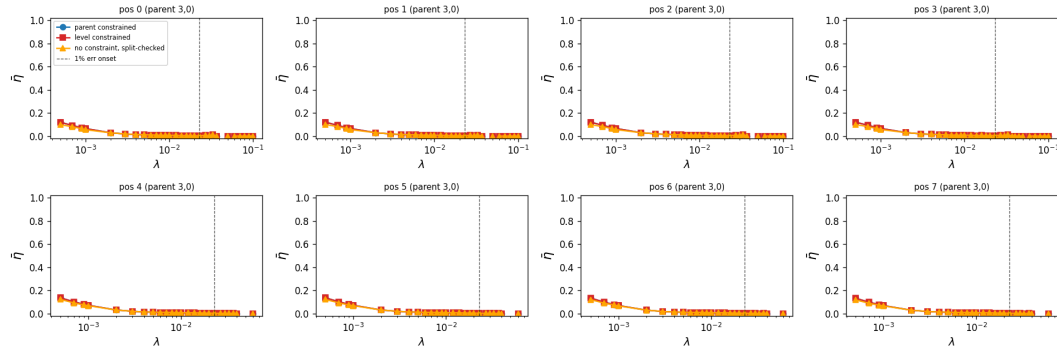


Figure 23: **Entropy curves for SAE₃ (root level), all token positions.** Same configuration. Root constrained collapses to zero entropy well before the onset of error.

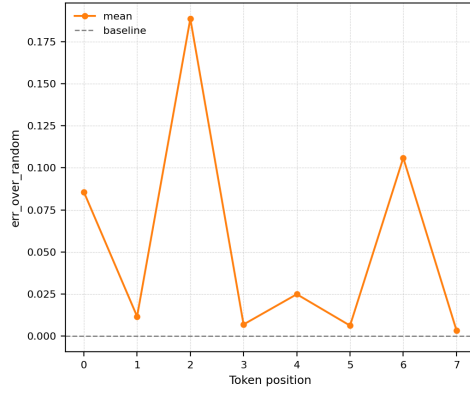


Figure 24: **Per-token zero-ablation after Block₁**. Same configuration. Odd positions 1, 3, 5, 7 sit at the baseline and have no effect when ablated. The figure isn't as regular as for the other instances, but the dead-token phenomenon is present here, even though the SAE₁ entropy of Fig. 21 does not reveal it.

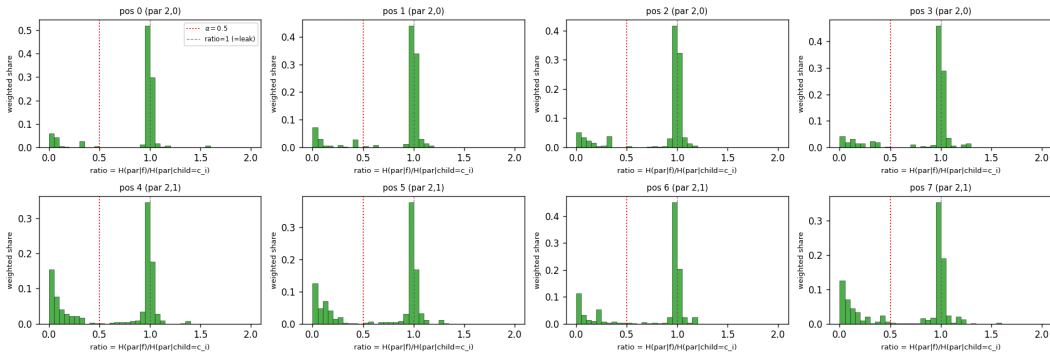


Figure 25: **Entropy ratio histogram for SAE₂, $\lambda = 0.023$** . Same configuration. The distributions are bimodal with a clean valley at $\alpha = 0.5$, and the absorbed features are correctly labeled ones.

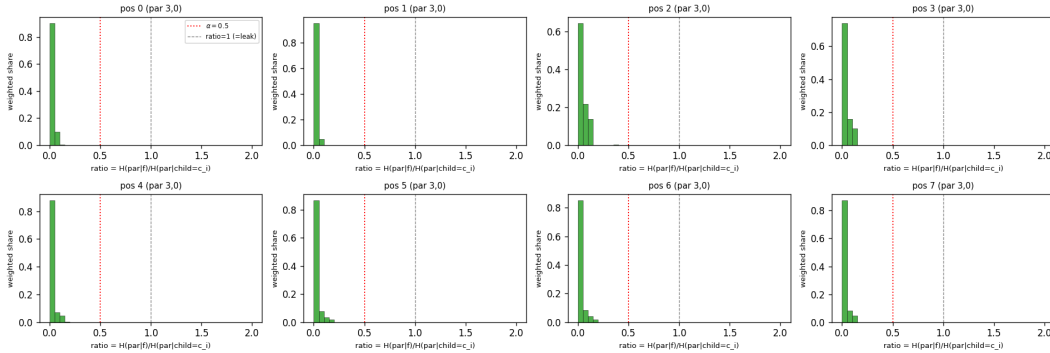


Figure 26: **Entropy ratio histogram for SAE₃ (root), $\lambda = 0.018$** . Same configuration. Almost all child-labeled features have ratio well below 0.5, so they pin the root more tightly than their child value warrants and are reassigned to the root. We can't compute the ratio for the features that were directly assigned to the root, which is why we don't see anything in the ratio ≈ 1 area.

D.2 $v = 16, L = 3, m = 4$, residual stream

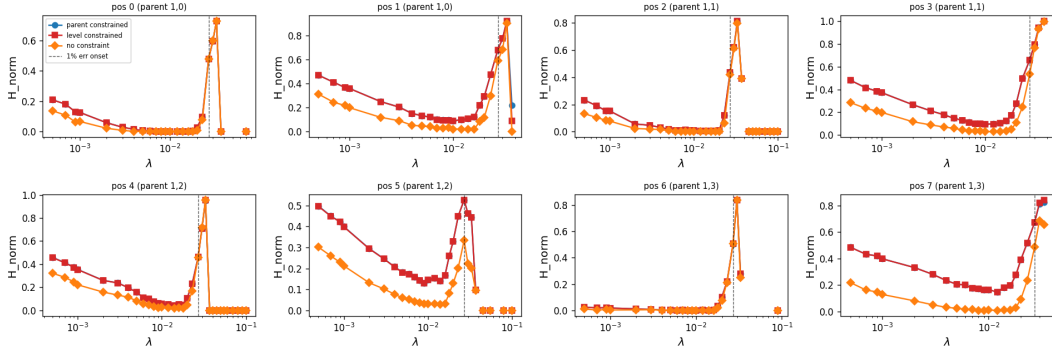


Figure 27: Entropy curves for SAE₁, all token positions. RHM parameters $v = 16, L = 3, m = 4$, with residual stream.

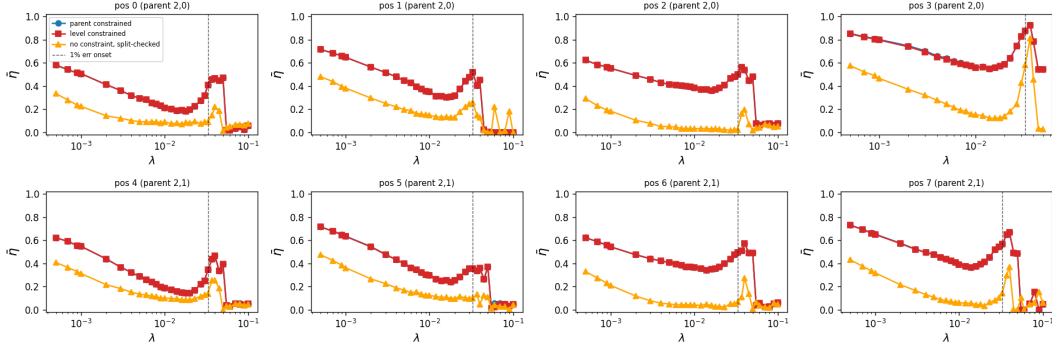


Figure 28: Entropy curves for SAE₂, all token positions. RHM parameters $v = 16, L = 3, m = 4$, with residual stream. The level-constrained curve stays clearly above the unconstrained one, so unconstrained labeling wins and the layer carries genuine lower-level features, as in the $m = 16$ residual variant.

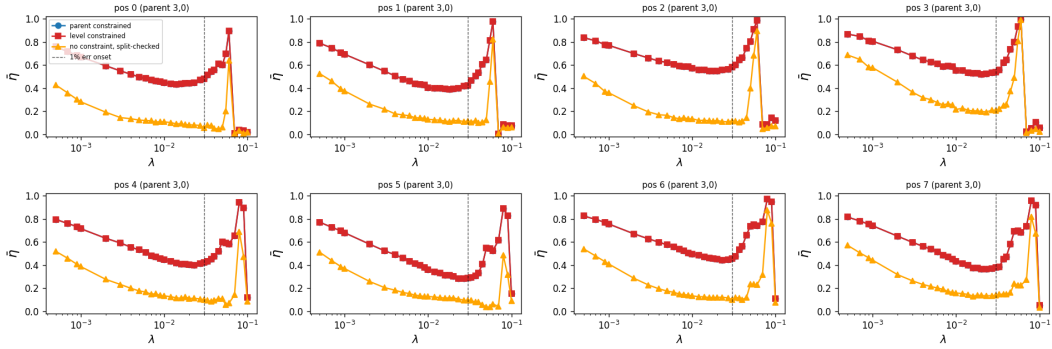


Figure 29: Entropy curves for SAE₃ (root), all token positions. Same configuration. The unconstrained curve again sits below the root-constrained one, with a spike at the onset, so lower-level information persists into the last block through the skip connections.

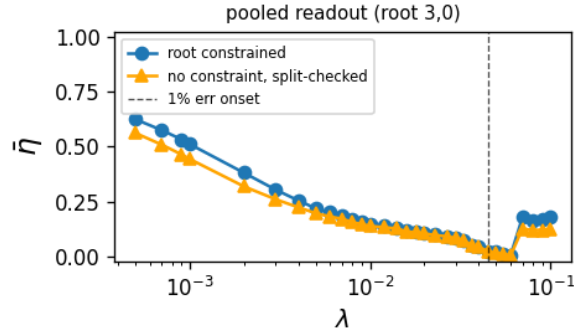


Figure 30: **Entropy curve for the pooled-readout SAE (root).** Same configuration. The root-constrained and unconstrained curves almost coincide and reach about zero just before the onset, so the pooled representation gives a much cleaner root-selective dictionary than the representations after SAE_3 .

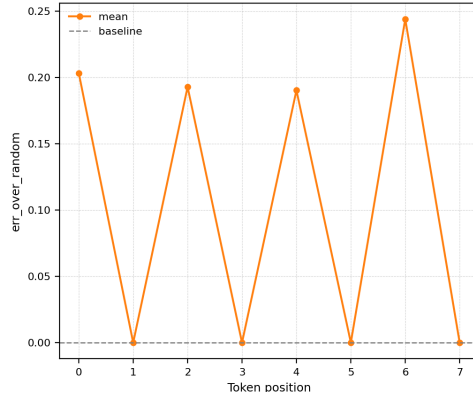


Figure 31: **Per-token zero-ablation after Block₁.** Same configuration. Odd positions 1, 3, 5, 7 have exactly zero effect.

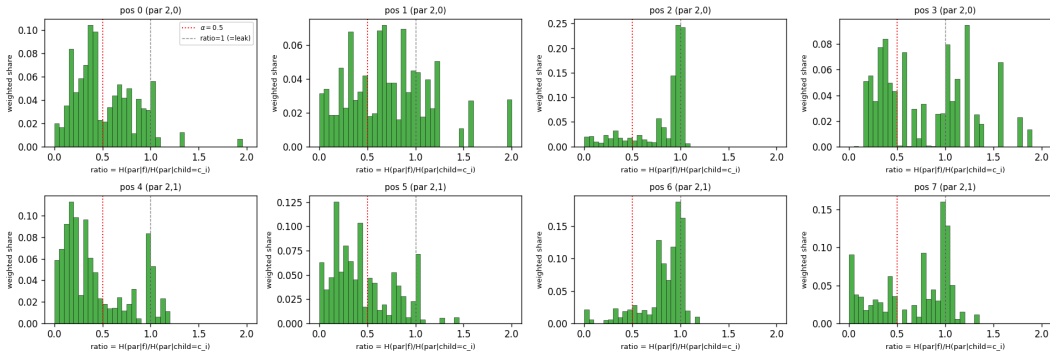


Figure 32: **Entropy ratio histogram for SAE_2 , $\lambda = 0.023$.** Same configuration. No clear valley, so no threshold separates genuine from absorbed features, the same failure as the $m = 16$ residual variant.

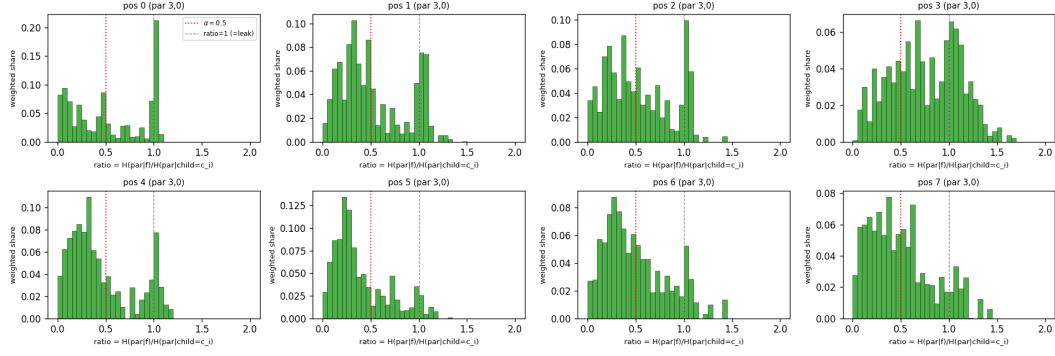


Figure 33: **Entropy ratio histogram for SAE₃ (root), $\lambda = 0.023$.** Same configuration. The mass is again spread on both sides of 0.5 with no clean threshold.

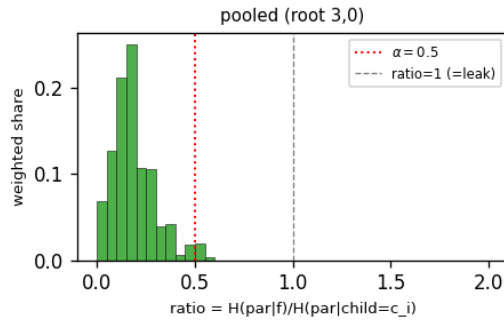


Figure 34: **Entropy ratio histogram for the pooled SAE, $\lambda = 0.033$.** Same configuration. A single mode below 0.5 shows that the features labeled to lower levels are absorbed and reassigned to the root, confirming the pooled dictionary is clean.

D.3 $v = 8, L = 4, m = 8$, no residual stream

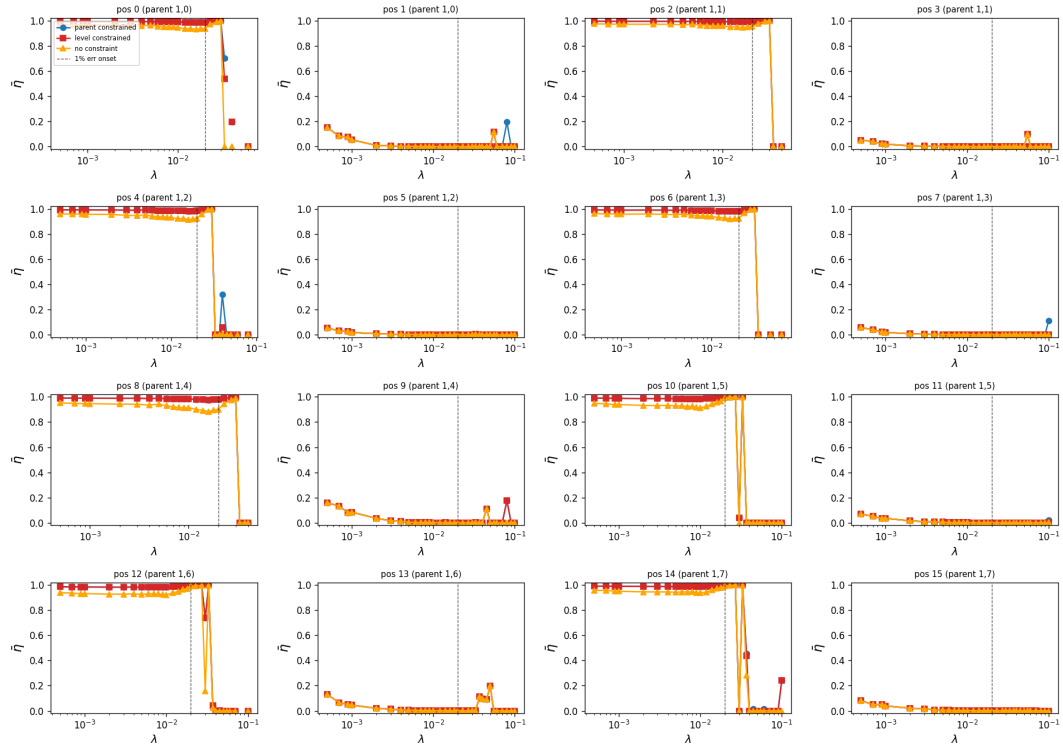


Figure 35: **Entropy curves for SAE₁, all token positions.** RHM parameters $v = 8, L = 4, m = 8$, no residual stream. Even positions $0, 2, \dots, 14$ stay near $\eta = 1$ under every constraint across the whole sweep, and correspond to the dead token positions.

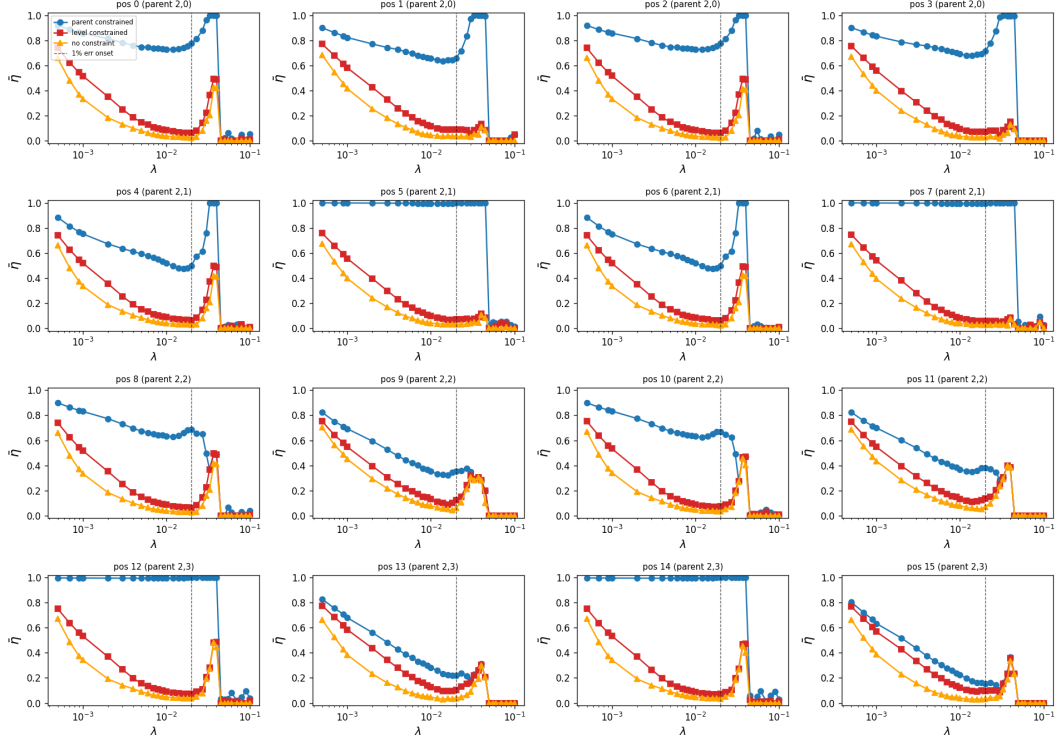


Figure 36: **Entropy curves for SAE₂, all token positions.** Same configuration. The clear ordering of parent-above-level curves shows delocalization. No significant difference in performance between level-constrained and unconstrained, so mainly absorbed features.

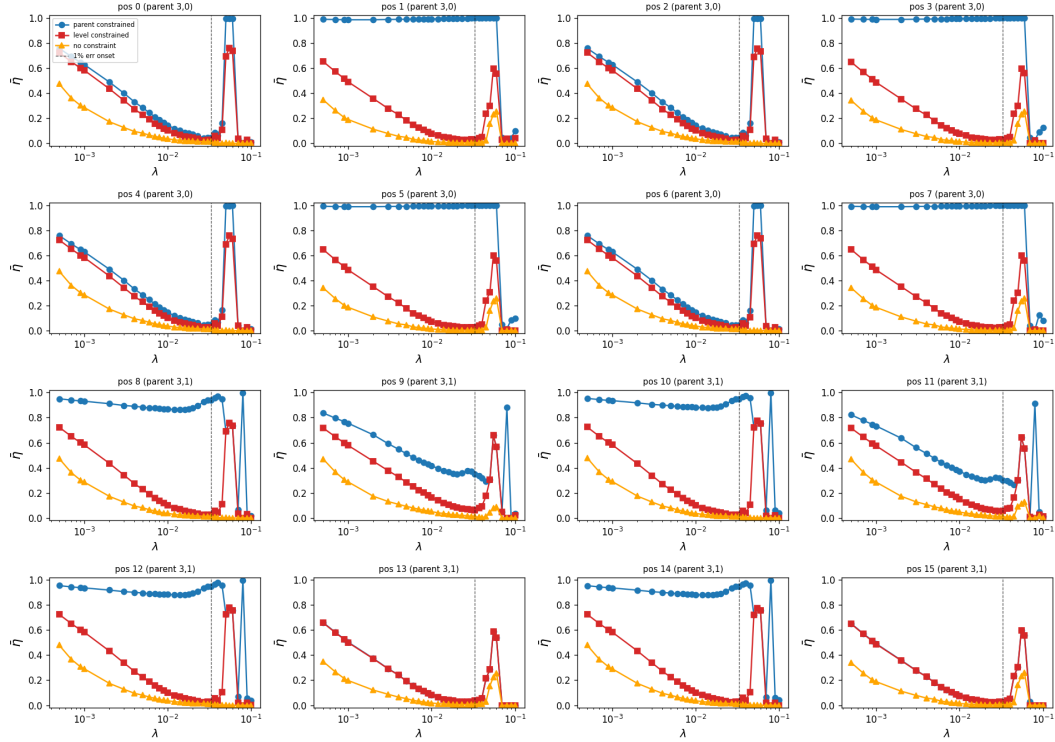


Figure 37: **Entropy curves for SAE₃, all token positions.** Same configuration. Same ordering of curves: delocalization is present. Level-constrained curves close to 0 before the onset of error, so level-correspondence seems to hold.

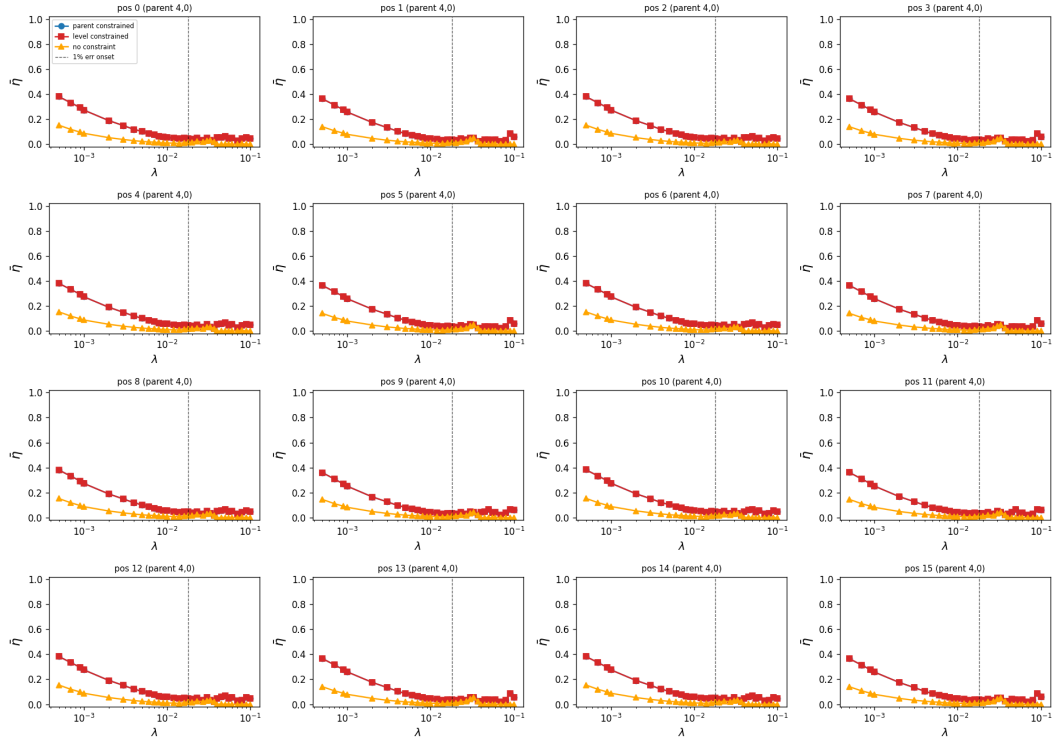


Figure 38: **Entropy curves for SAE₄ (root level), all token positions.** Same configuration. The root-constrained and unconstrained curves both fall close to 0 before the onset of error; features are pinning down the root (also see Fig. 42).

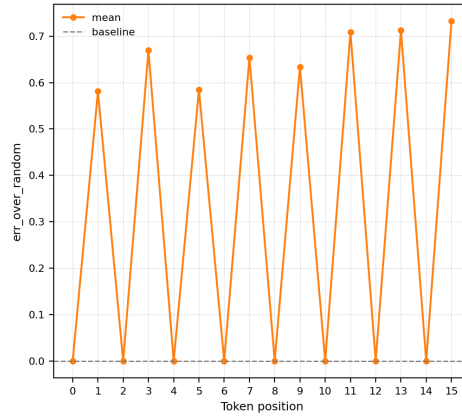


Figure 39: **Per-token zero-ablation after Block₁.** Same configuration. Even positions 0, 2, $\dots$, 14 sit at the baseline, illustrating the dead-token phenomenon.

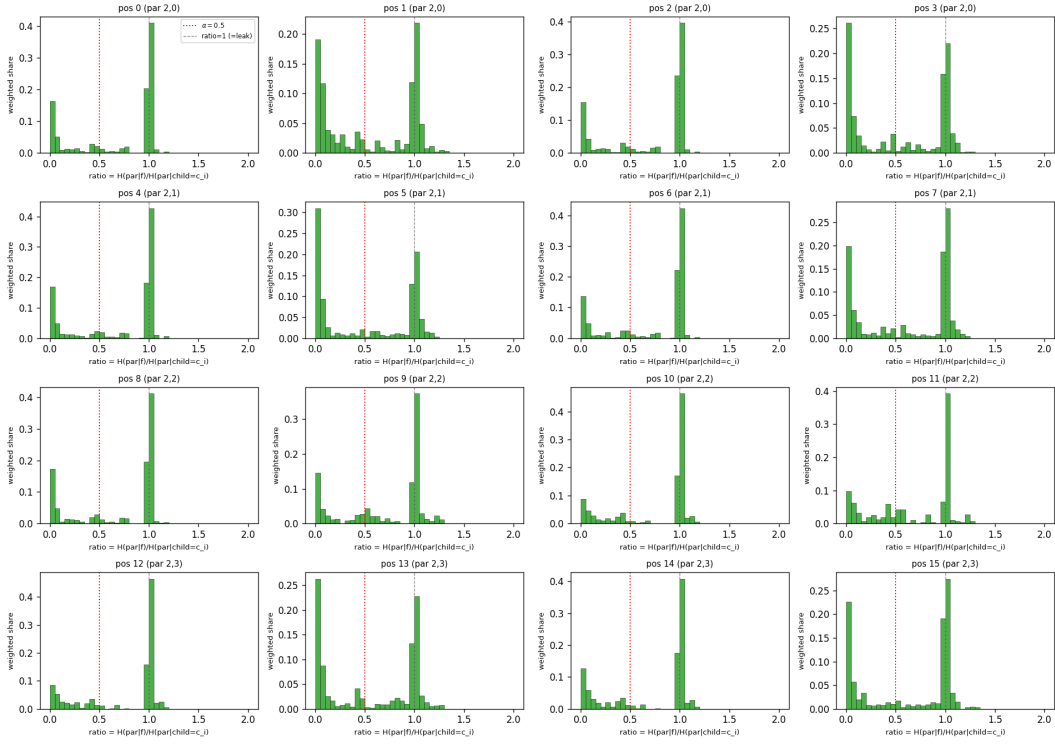


Figure 40: **Entropy ratio histogram for SAE₂, $\lambda = 0.02$.** Same configuration. The distribution is bimodal with a valley near 0.5. Absorbed and genuine features separate cleanly.

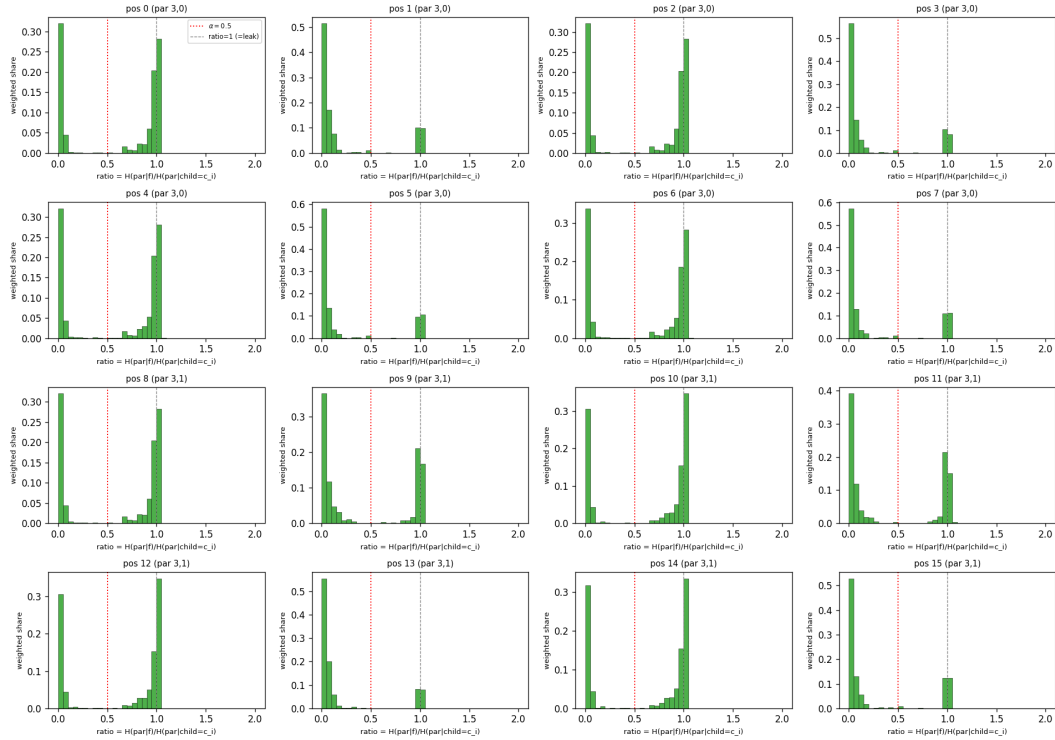


Figure 41: **Entropy ratio histogram for SAE₃, $\lambda = 0.03$.** Same configuration. The distribution is again bimodal, giving the same clean separation one layer deeper.

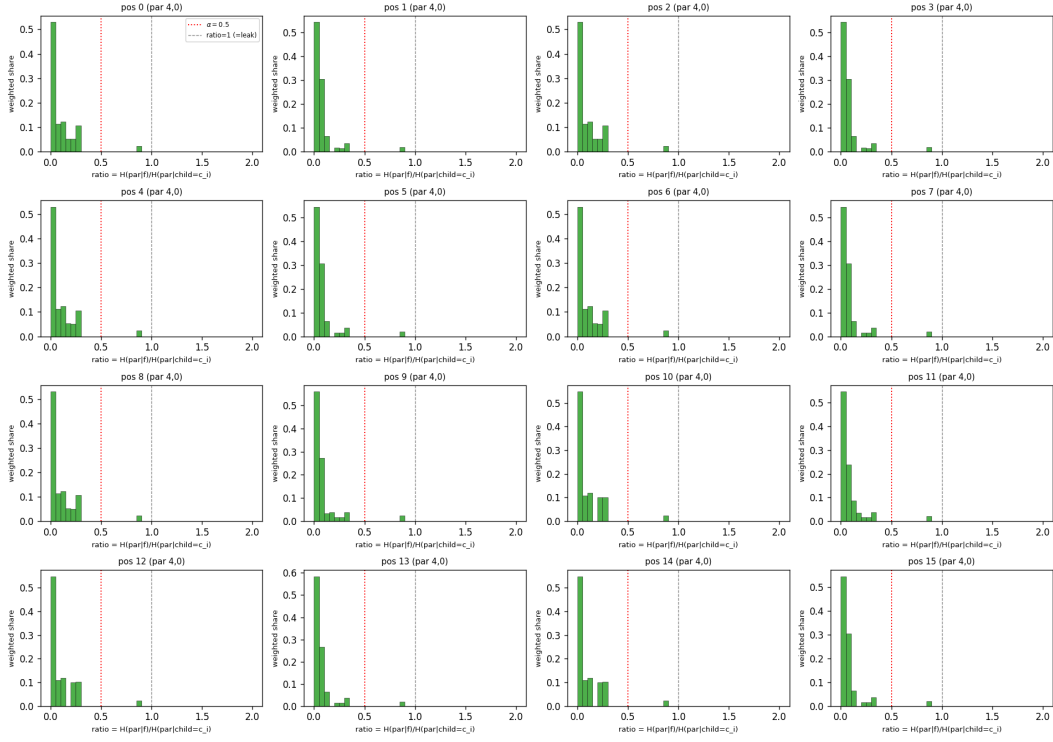


Figure 42: **Entropy ratio histogram for SAE₄ (root), $\lambda = 0.023$.** Same configuration. A single mode near 0 shows that the child-labeled features are all absorbed root features and are reassigned to the root.

D.4 $v = 8, L = 4, m = 8$, residual stream

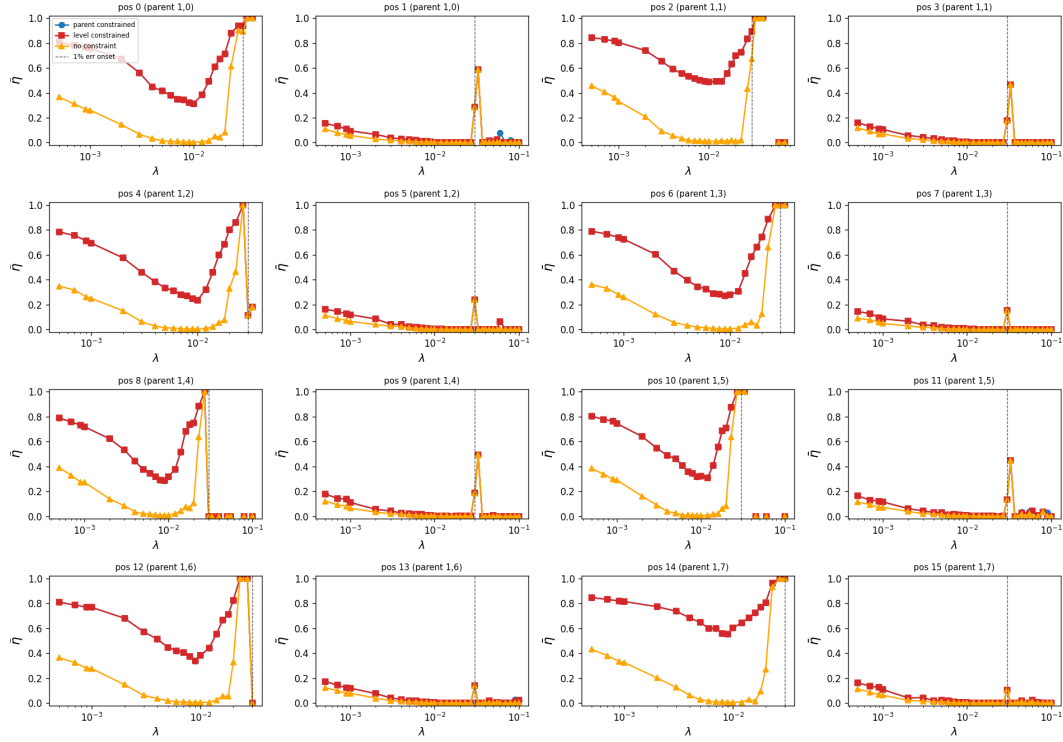


Figure 43: **Entropy curves for SAE₁, all token positions.** RHM parameters $v = 8, L = 4, m = 8$, with residual stream. Even positions have high entropy under level constraint, meaning they contain more leaf features. These correspond to the dead tokens showcased by Fig. 48. Parent overlaps level, so locality holds.

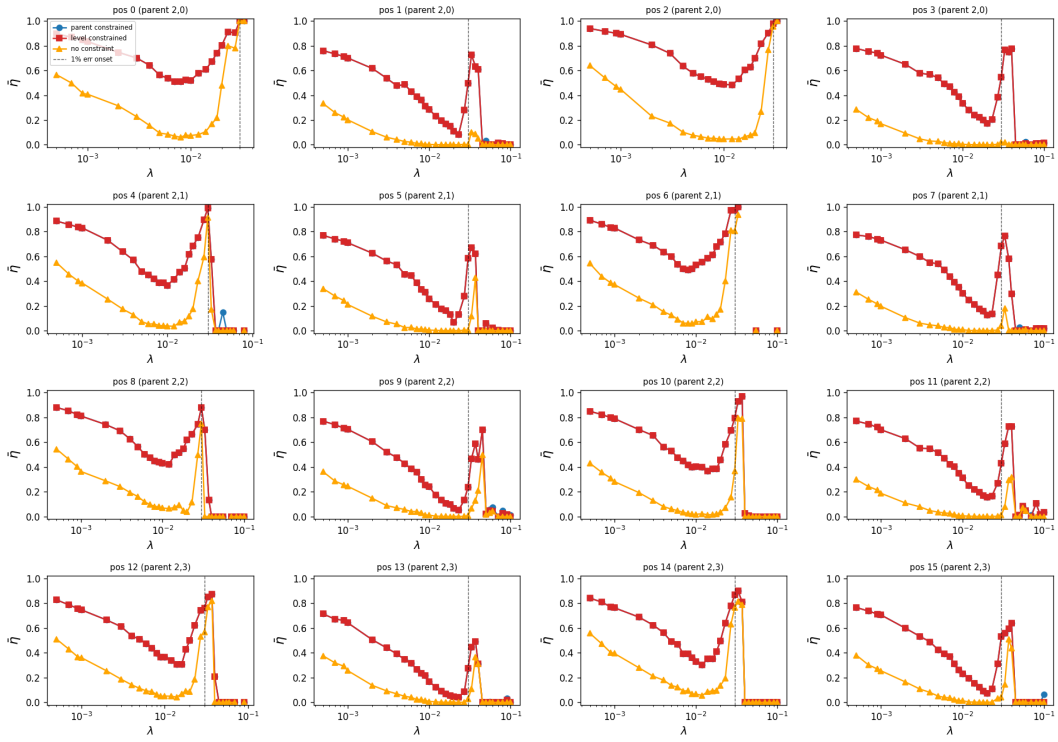


Figure 44: **Entropy curves for SAE₂, all token positions.** Same configuration. The unconstrained curves lie clearly below the level-constrained one for the even positions, less so for the odd positions. These odd positions seem to contain corresponding level 2 features, which seems confirmed by Fig. 49.

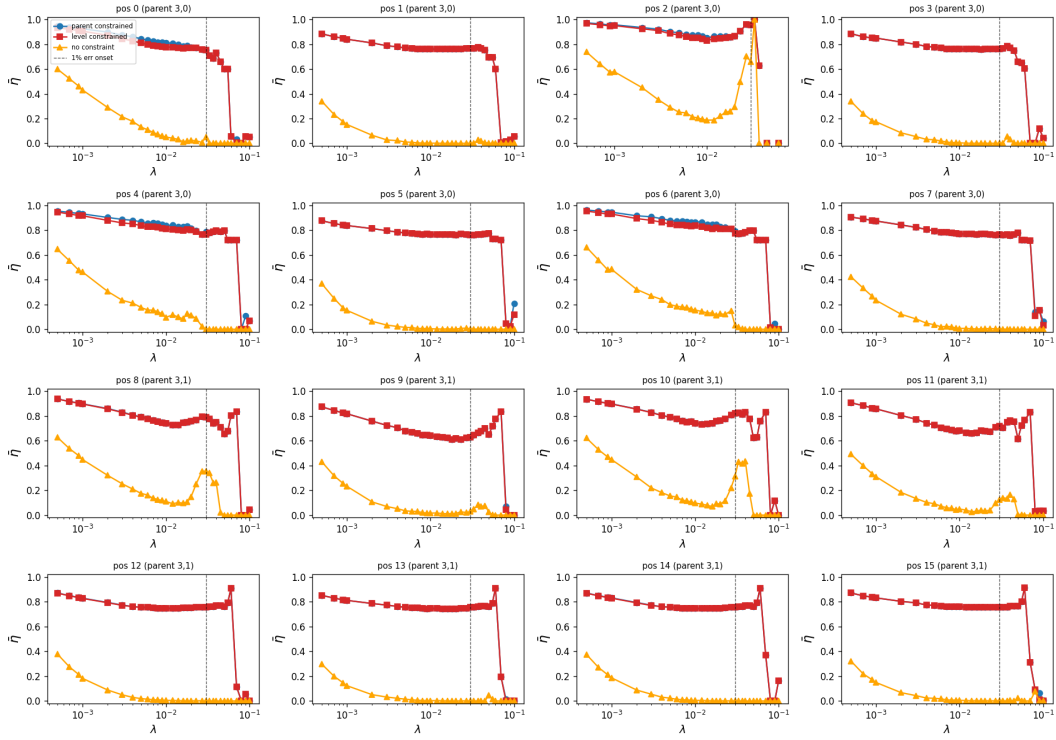


Figure 45: **Entropy curves for SAE₃, all token positions.** Same configuration. The wide gap between unconstrained and level-constrained indicates lower-level information is pushed deep by the skip connections.

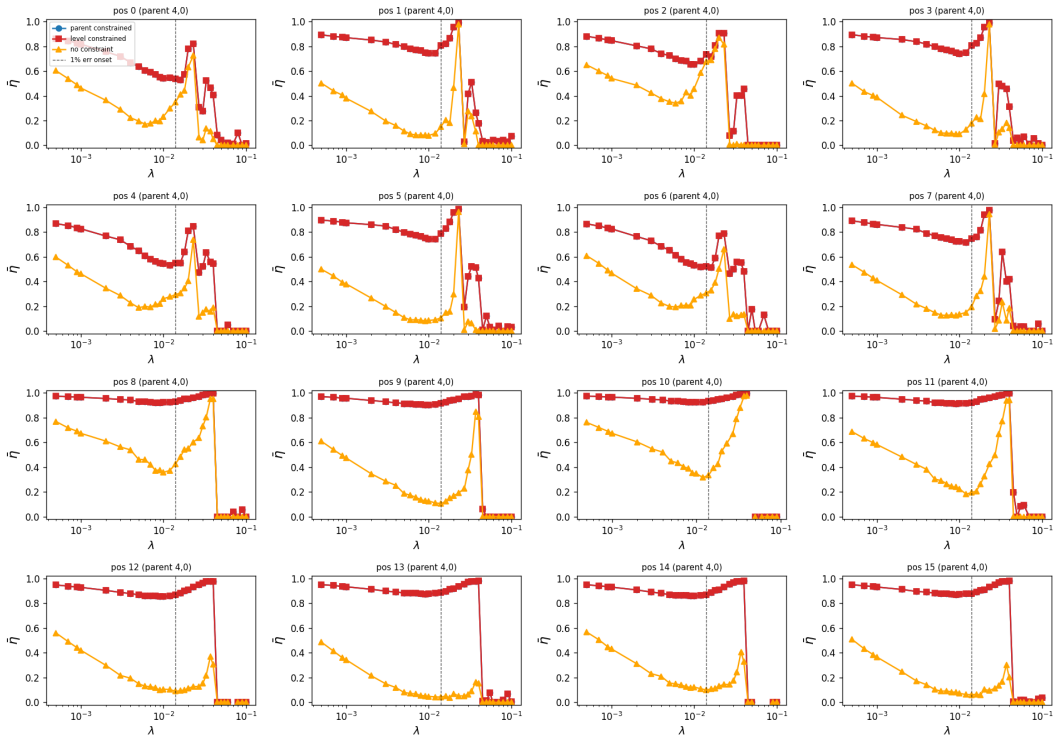


Figure 46: **Entropy curves for SAE₄ (root level), all token positions.** Same configuration. Same wide gap between unconstrained and level-constrained signaling lower-level features.

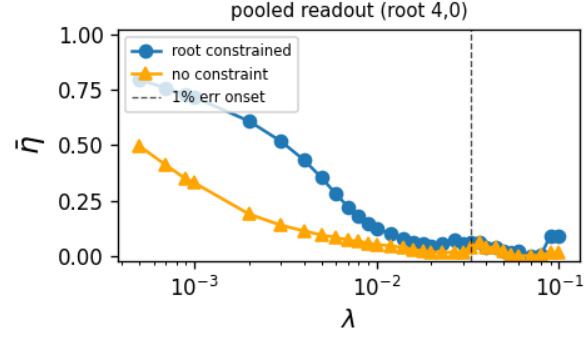


Figure 47: **Entropy curve for the pooled-readout SAE (root).** Same configuration. The root-constrained and unconstrained curves nearly coincide close to the onset of error, so the pooled dictionary is root-selective.

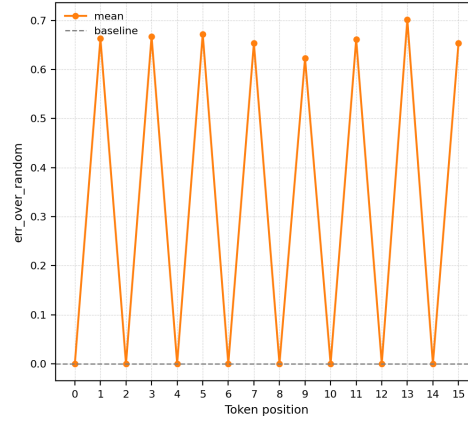


Figure 48: **Per-token zero-ablation after Block₁.** Same configuration. Even positions 0, 2, ..., 14 sit at the baseline, so the dead-token phenomenon is present with one live position per pair.

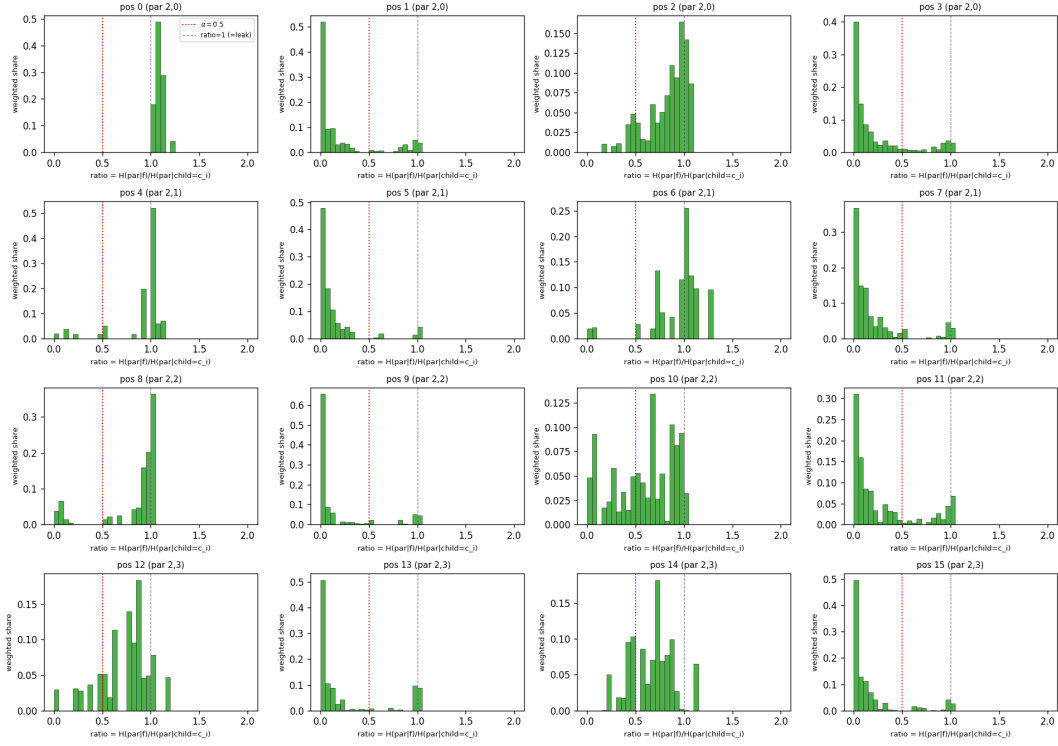


Figure 49: **Entropy ratio histogram for SAE₂, $\lambda = 0.02$.** Same configuration. Some distributions seem to cleanly distinguish genuine and absorbed features.

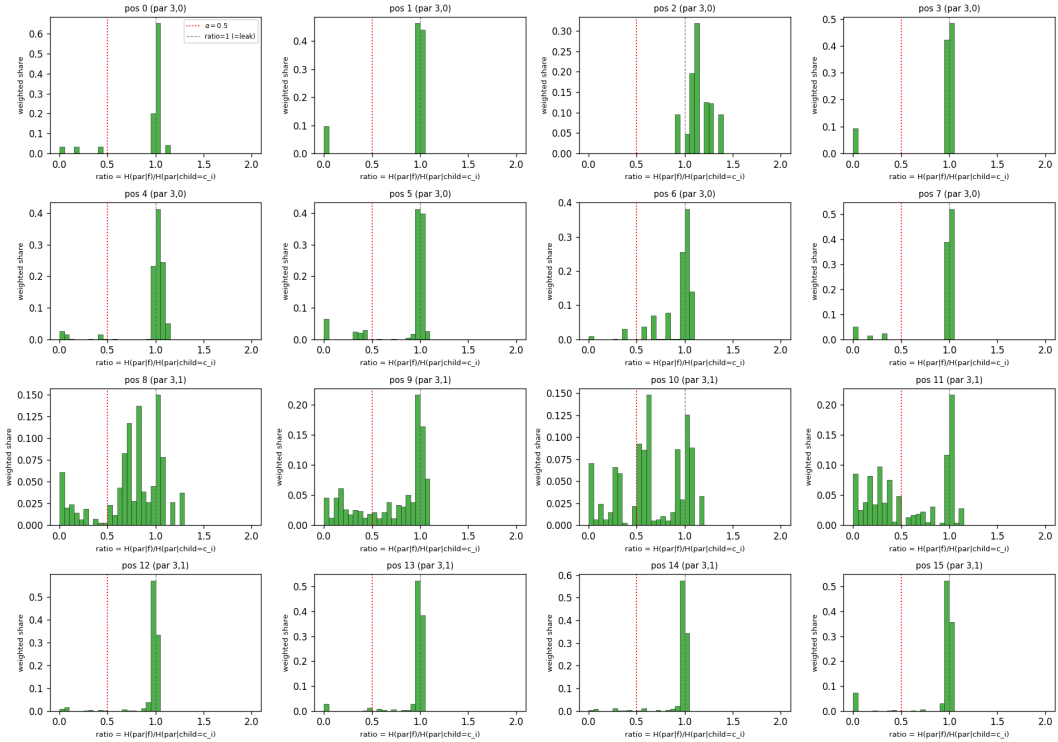


Figure 50: **Entropy ratio histogram for SAE₃, $\lambda = 0.023$.** Same configuration. Most of the positions still have distributions cleanly separating genuine from absorbed features.

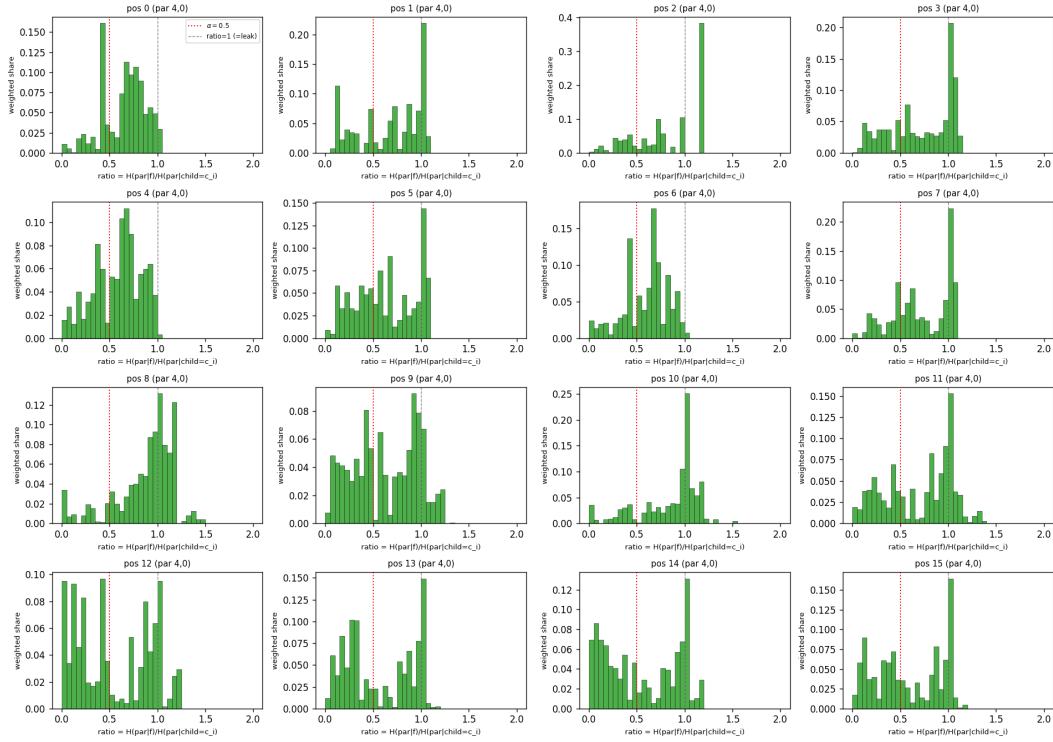


Figure 51: **Entropy ratio histogram for SAE₄ (root), $\lambda = 0.012$.** Now, most positions don't showcase a clear distinction between genuine and absorbed features.

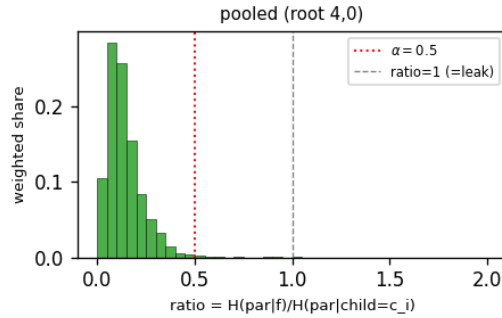


Figure 52: **Entropy ratio histogram for the pooled SAE, $\lambda = 0.01$.** Same configuration. A clean single mode below 0.5 shows that the features that were assigned to non-root latents correspond to absorbed features.